

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ
БЕЛАРУСЬ
Учреждение образования «БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет Информационных технологий
Кафедра Программной инженерии
Специальность 6-05-0612-01 «Программная инженерия»
Направление специальности 6-05-0612-01 «Программная инженерия»

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
КУРСОВОГО ПРОЕКТА:

по дисциплине «Объектно-ориентированные технологии программирования
и стандарты проектирования»

Тема Программное средство «Агентство недвижимости»

Исполнитель

Студент(ка) 2 курса группы 10 Бакуменко Данила Владимирович
(Ф.И.О.)

Руководитель работы ст.преп. Сухорукова И. Г.
(учен. степень, звание, должность, подпись, Ф.И.О.)

Курсовой проект защищен с оценкой

Председатель Белодед Н. И.
(подпись)

Минск 2026

Оглавление

Введение	5
1. Постановка задачи и обзор литературы	6
1.1 Обзор аналогов	6
1.1.1 Анализ сервиса «Яндекс Недвижимость»	6
1.1.2 Анализ сервиса «Домклик»	9
1.1.3 Анализ сервиса «Cian.ru»	11
1.2 Постановка задачи	13
2 Анализ требований к программному средству и разработка функциональных требований	15
2.1 Описание средств разработки	15
2.2 Описание разрабатываемой функциональности программного средства	17
2.2.1 Пользовательские роли в системе	17
2.2.2 Функциональные требования	18
2.2.2.1 Общие функции системы	18
2.2.2.2 Функции администратора	18
2.2.2.3 Функции риелтора	18
2.2.2.4 Функции клиента	19
2.2.3 Диаграмма вариантов использования	19
2.2.4 Спецификация функциональных требований	19
2.2.4.1 Требования к регистрации и авторизации	19
2.2.4.2 Требования к работе с объектами недвижимости	20
2.2.4.3 Требования к уведомлениям и отзывам	20
2.2.4.4 Требования к администрированию и отчетности	20
3 Проектирование программного средства	22
3.1 Архитектура системы	22
3.2 Диаграмма компонентов	23
3.3 Взаимоотношения между классами	23
3.4 Проектирование модели базы данных	23
3.5 Проектирование последовательности взаимодействия	25
4 Реализация программного средства	26
4.1 Реализация архитектуры проекта на основе MVVM	26
4.2 Реализация ключевых функций	27
4.2.1 Реализация функции авторизации и регистрации	27
4.2.2 Реализация функции добавления объекта недвижимости	28
4.2.3 Реализация функции оформления заявки на показ	28
4.2.4 Реализация системы многокритериальной фильтрации каталога	28
4.2.5 Реализация системы email-уведомлений	29
4.2.6 Реализация паттернов Repository и Unit of Work	29
4.3 Реализация модели данных	30
5 Тестирование, проверка работоспособности и анализ полученных результатов	32
5.1 Тестирование сервиса регистрации и авторизации	32

5.2 Тестирование функций риелтора	33
5.2.1 Тестирование функции добавления объекта недвижимости	33
5.2.2 Тестирование функции управления заявками	34
5.3 Тестирование функций клиента	34
5.3.1 Тестирование функции оформления заявки на показ	34
5.3.2 Тестирование функции публикации отзыва	35
5.3.3 Тестирование функции управления избранным	36
5.4 Тестирование функций администратора	36
5.4.1 Тестирование управления ролями пользователей	36
5.4.2 Тестирование функции модерации отзывов	37
6 Руководство по установке и использованию	38
6.1 Требования к аппаратному и программному обеспечению	38
6.2 Установка программного средства	38
6.3 Авторизация и регистрация	38
6.4 Руководство по использованию для клиента	40
6.5 Руководство по использованию для риелтора	43
6.6 Руководство по использованию для администратора	46
Заключение	49
Список использованных источников	50
Приложение А	51
Приложение Б	52
Приложение В	53
Приложение Г	54
Приложение Д	55
Приложение Е	56
Приложение Ж	57
Приложение З	58
Приложение И	60
Приложение К	62

Введение

В современном мире операции с недвижимостью – покупка, продажа или аренда жилья – являются одними из самых значимых и сложных процессов в жизни человека. Однако поиск подходящего объекта часто сопряжен с рядом трудностей: необходимостью ручного мониторинга сотен объявлений, риском столкнуться с неактуальными данными и сложностью оперативной организации осмотра объекта. Для клиентов критически важна возможность не просто просмотреть каталог, но и заблаговременно забронировать время для показа, чтобы обезопасить себя от ситуации, когда понравившийся объект будет продан или сдан другому лицу до момента встречи с риелтором.

Данное программное средство незаменимо для лиц, заинтересованных в операциях с жилой и коммерческой площадью, поскольку благодаря интуитивно понятному пользовательскому интерфейсу позволяет самостоятельно подобрать объект недвижимости, предварительно ознакомившись с подробными техническими характеристиками и медиафайлами, не выходя из дома.

Система предоставляет пользователю возможность удаленного взаимодействия с агентством. Создание учетной записи позволяет клиенту не только просматривать актуальный каталог с использованием гибкой системы фильтров (по цене, площади и другим параметрам), но и формировать список «Избранного» для последующего быстрого доступа к понравившимся вариантам. Одной из ключевых функций является возможность оформления онлайн-заявки на показ объекта в удобное для клиента время, что значительно сокращает цикл заключения сделки.

С точки зрения бизнес-процессов, программное средство является инструментом автоматизации работы сотрудников агентства. Риелторы получают возможность оперативно управлять базой объектов, загружать фотографии, а также вести расписание показов и отслеживать личную статистику эффективности.

Административная часть системы обеспечивает полный контроль над качеством предоставляемых услуг: управление штатом риелторов, модерацию клиентских отзывов и массовое оповещение пользователей через современные каналы связи (E-mail). Инструменты аналитики позволяют руководству отслеживать статистические показатели работы агентства в режиме реального времени.

Главная задача данного курсового проектирования – разработка программного средства для агентства недвижимости, которое реализует все вышеперечисленные функции и обеспечивает эффективное взаимодействие между клиентом, риелтором и администрацией сервиса.

1. Постановка задачи и обзор литературы

1.1 Обзор аналогов

В данном разделе будет проведен обзор и анализ существующих веб-ресурсов и информационных систем, работающих в сфере недвижимости, с целью выявления их особенностей, функциональных возможностей и уровня удобства для различных категорий пользователей (клиентов, риелторов и администраторов).

Этот анализ является ключевым этапом перед постановкой задачи на разработку программного средства «Агентство недвижимости». Изучение существующих решений позволит понять, какие инструменты являются стандартными для рынка, а какие требуют доработки для повышения эффективности бизнес-процессов агентства.

Основные критерии, которые будут учитываться при анализе, включают функциональность платформы (полнота фильтров, удобство управления базой данных, наличие инструментов обратной связи), эргономику интерфейса и качество реализации личных кабинетов для разных ролей. Сравнив существующие системы, мы сможем определить сильные и слабые стороны конкурентов и на основе этого спроектировать более эффективный, надежный и удобный программный продукт, отвечающий современным требованиям автоматизации сделок с недвижимостью.

1.1.1 Анализ сервиса «Яндекс Недвижимость»

Сервис «Яндекс Недвижимость» является одним из лидеров рынка в сфере цифровых услуг по поиску и продаже жилья. В рамках анализа рассматривались возможности системы по подбору объектов, информативность карточек и механизмы взаимодействия пользователя с платформой.

Система поиска и фильтрации. Программное средство обладает мощным инструментом технического поиска (рисунок 1.1).

Рисунок 1.1 – Интерфейс расширенных фильтров «Яндекс Недвижимость»

Пользователю доступна глубокая настройка параметров: от стандартных (цена, площадь, этаж) до узкоспециализированных, таких как площадь кухни, высота потолков, наличие мебели или возможность просмотра 3D-тура. Такая детализация позволяет минимизировать время на просмотр нерелевантных объектов. Представление каталога и карточки объекта. Результаты поиска отображаются в виде структурированной сетки (рисунок 1.2). Каждая карточка содержит ключевую информацию: цену, статус новостройки и удаленность от метро. При переходе к детальному изучению (рисунок 1.3) пользователь получает доступ к фотогалерее, 3D-визуализации и блоку с ценовыми диапазонами. Информация о застройщике и статусе объекта вынесена в отдельный информационный блок, что упрощает восприятие данных.

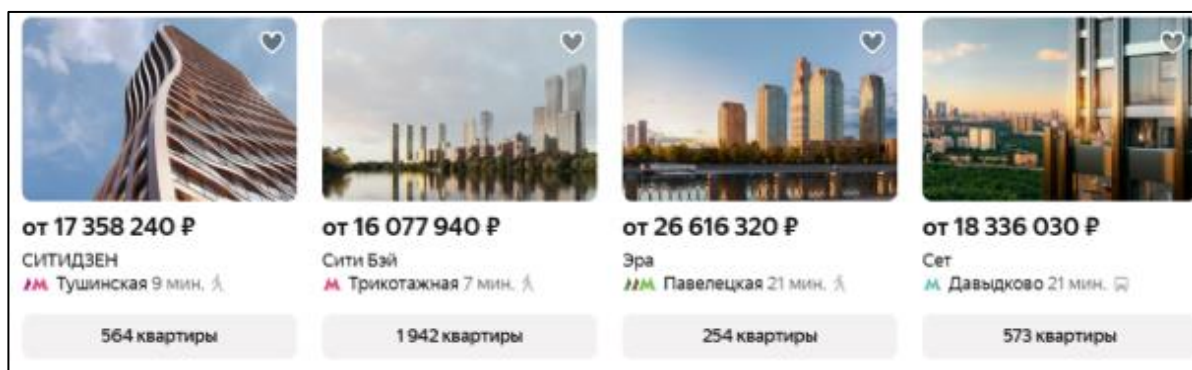


Рисунок 1.2 – Каталог объектов в виде сетки

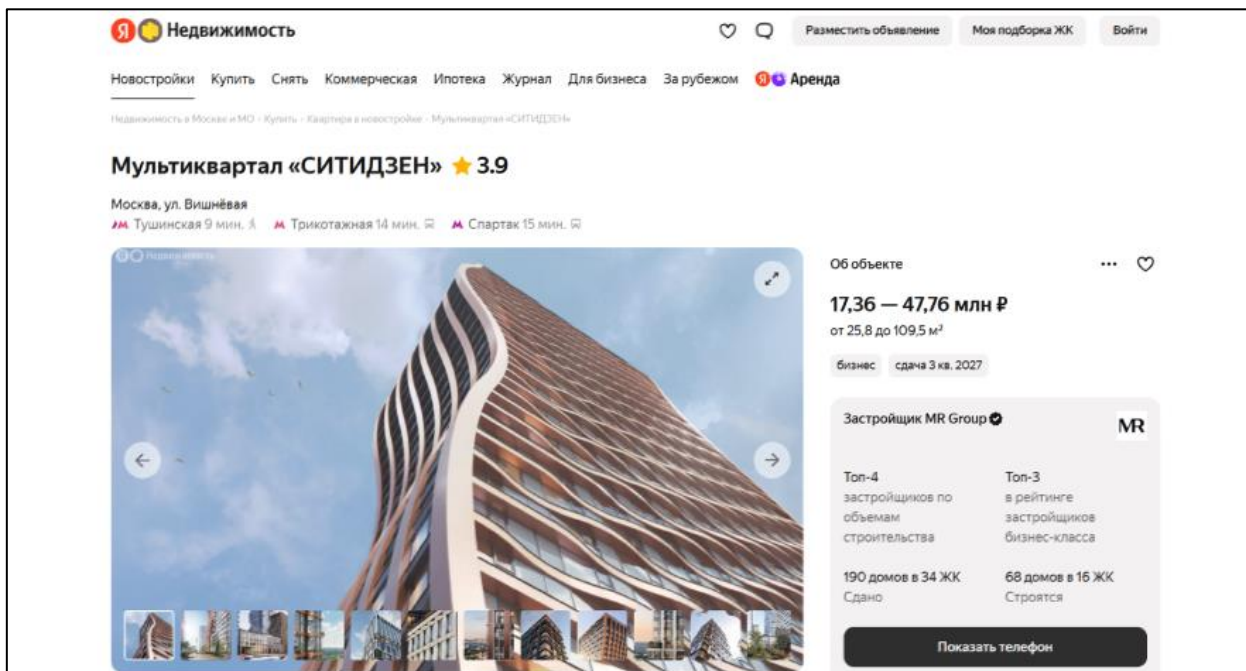


Рисунок 1.3 – Детальная карточка объекта недвижимости

Личный кабинет и инструменты лояльности. Система поддерживает функции регистрации и авторизации. В личном кабинете реализован раздел «Избранное» (рисунок 1.4), где пользователь может сохранять понравившиеся варианты. Важной особенностью является наличие кнопок прямого

взаимодействия: «Показать телефон» и «Позвоните мне», что позволяет клиенту оперативно связаться с представителем агентства. Управление личными данными и документами интегрировано с единой системой авторизации (рисунок 1.5), что обеспечивает безопасность и удобство хранения профильной информации.

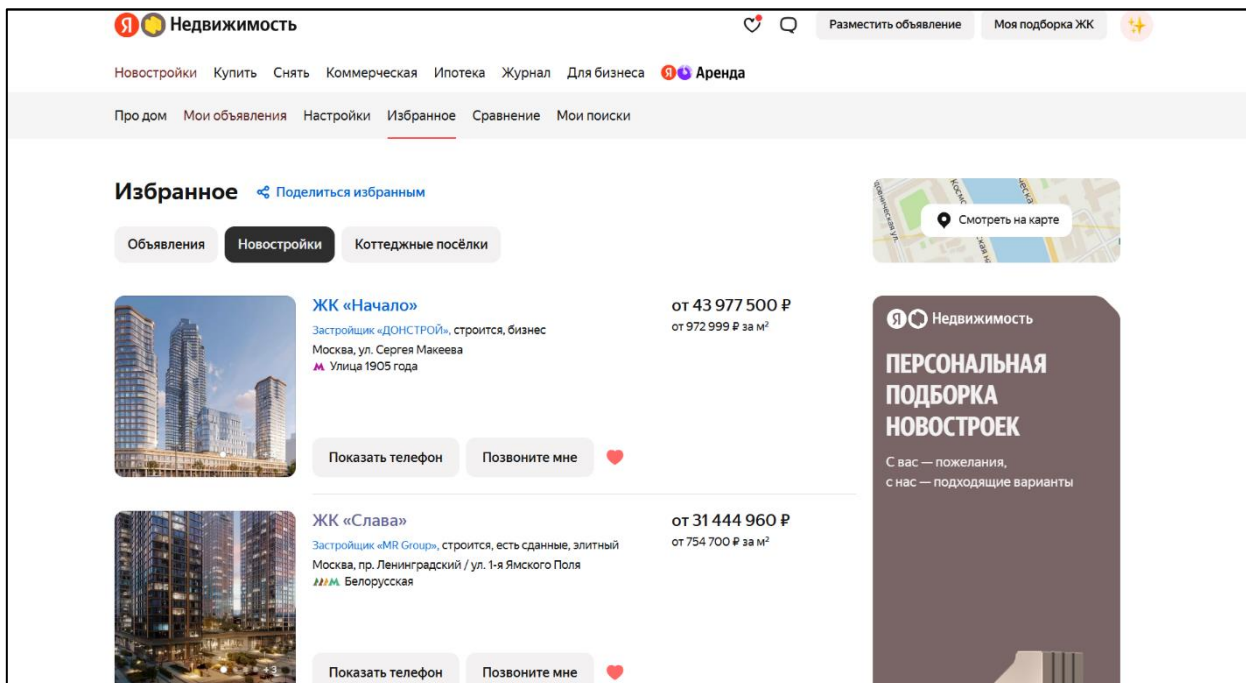


Рисунок 1.4 – Раздел «Избранное» с инструментами связи

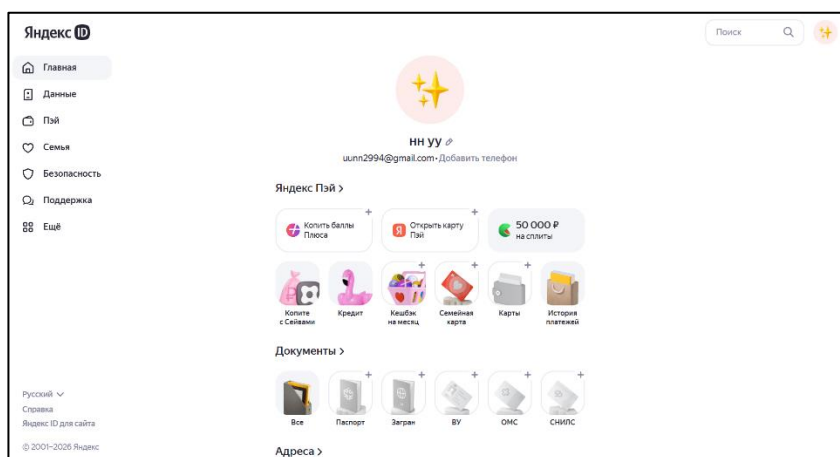


Рисунок 1.5 – Личный кабинет пользователя и управление данными

Обратная связь и валидация. Программное средство предоставляет возможность публикации и просмотра отзывов о конкретных жилых комплексах (рисунок 1.6). Пользователь видит не только сухие характеристики объекта, но и реальный опыт взаимодействия других людей с застройщиком или агентством. Наличие рейтинга (по 5-балльной шкале) и живых комментариев от реальных людей позволяет пользователю составить объективное мнение об объекте и работе посредников до начала очных переговоров.

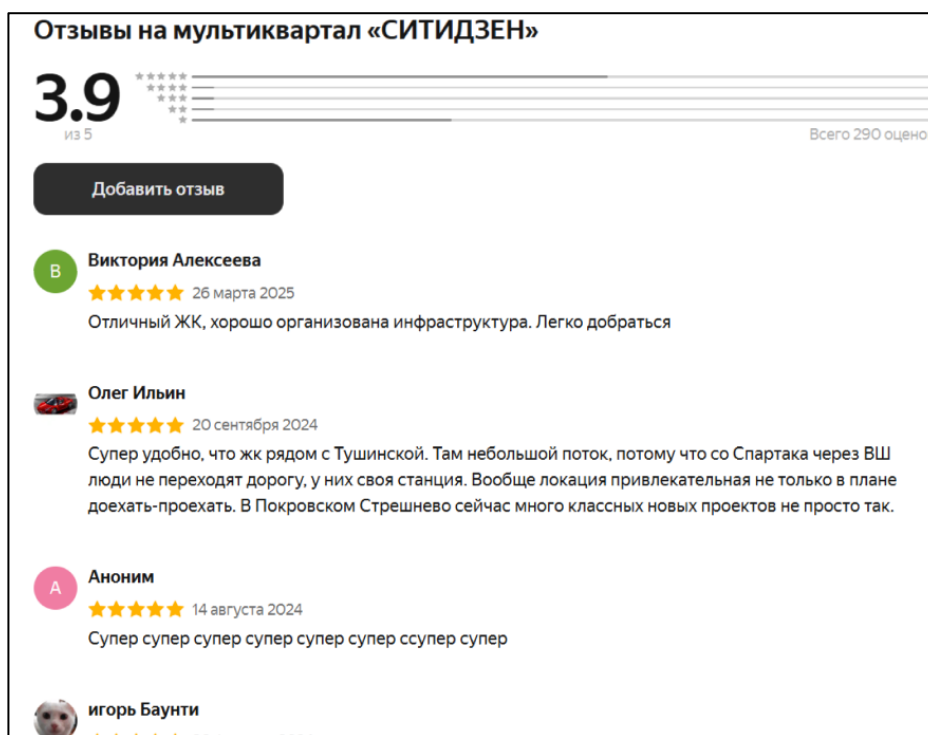


Рисунок 1.6 – Система отзывов и рейтингов объектов

1.1.2 Анализ сервиса «Домклик»

Информационная система «Домклик» [2] представляет собой пример платформы с глубокой проработкой технических характеристик объектов недвижимости. В рамках обзора были проанализированы механизмы поиска, структура представления каталога и наполнение карточек объектов.

Инструменты технического поиска. Одной из наиболее значимых особенностей системы является детализированная панель фильтров (рисунок 1.7).

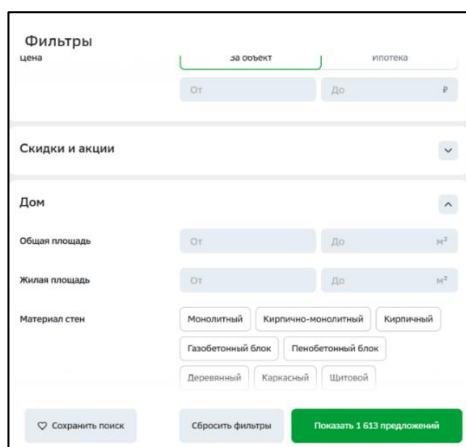


Рисунок 1.7 – Панель технических фильтров сервиса «Домклик»

Помимо стандартных параметров (цена и площадь), программное средство позволяет пользователю производить выбор по конструктивным особенностям здания, таким как материал стен (монолитный, кирпичный,

газобетонный блок и др.). Реализация фильтров по типу ипотечных программ и акций позволяет пользователю сразу оценить финансовую сторону сделки, не переходя к полному описанию объекта.

Организация каталога объектов. Представление базы данных в системе реализовано в виде плиточной сетки с акцентом на визуальную составляющую (рисунок 1.8). Каждое объявление в списке снабжено информативными тегами (например, «Мгновенное бронирование»), а также краткой выжимкой: адресом, ценой за объект и ценой за период (в случае аренды). Такая структура позволяет пользователю проводить первичный сравнительный анализ нескольких вариантов непосредственно в общем списке выдачи.

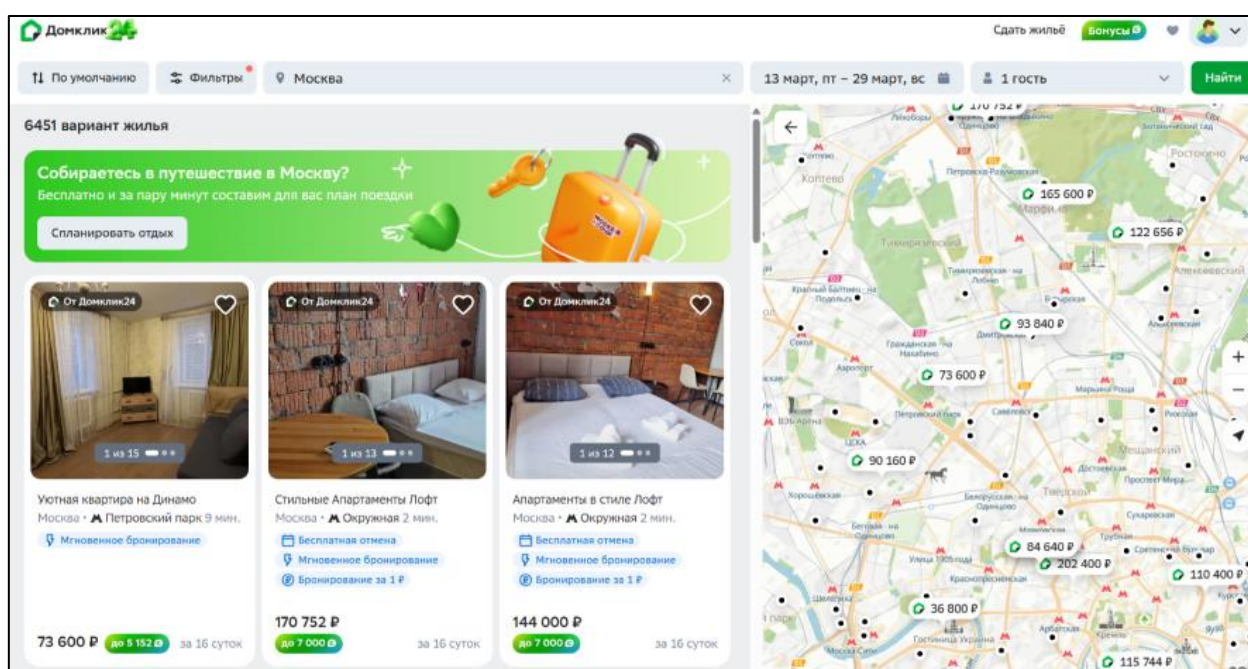


Рисунок 1.8 – Представление каталога недвижимости в виде сетки

Карточка объекта и инструменты связи. Карточка конкретной недвижимости (рисунок 1.9) содержит исчерпывающий набор данных для ознакомления клиента: общую площадь, размер участка, целевое назначение земли и подробное текстовое описание.

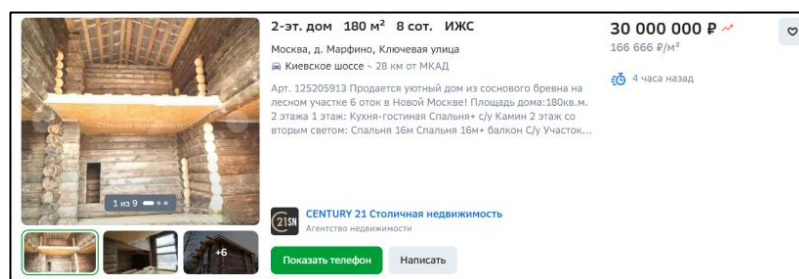


Рисунок 1.9 – Информационное наполнение карточки объекта недвижимости

Важным функциональным элементом является блок идентификации продавца, где указывается название агентства и имя ответственного риелтора. Для реализации прямой коммуникации предусмотрены закрепленные кнопки

действия: «Показать телефон» и «Написать», что обеспечивает быстрый переход пользователя от ознакомления к уточнению деталей.

1.1.3 Анализ сервиса «Cian.ru»

Информационный ресурс «Циан» [3] является одним из старейших агрегаторов на рынке недвижимости, предлагающим пользователям обширную базу данных по аренде и продаже жилых и коммерческих объектов. В ходе анализа были изучены механизмы первичного подбора, визуализация списка объектов и информативность карточек объявлений.

Система фильтрации и навигации. Сервис предлагает широкий выбор категорий недвижимости (квартиры, комнаты, дома, гаражи), представленный в виде выпадающих списков (рисунок 1.10). Однако, несмотря на разнообразие параметров, интерфейс системы фильтрации характеризуется избыточностью элементов и сложной логикой взаимодействия. Навигация по расширенным фильтрам затруднена из-за нагромождения текстовых полей и мелких чекбоксов, что снижает общую эргономичность поиска для неподготовленного пользователя.

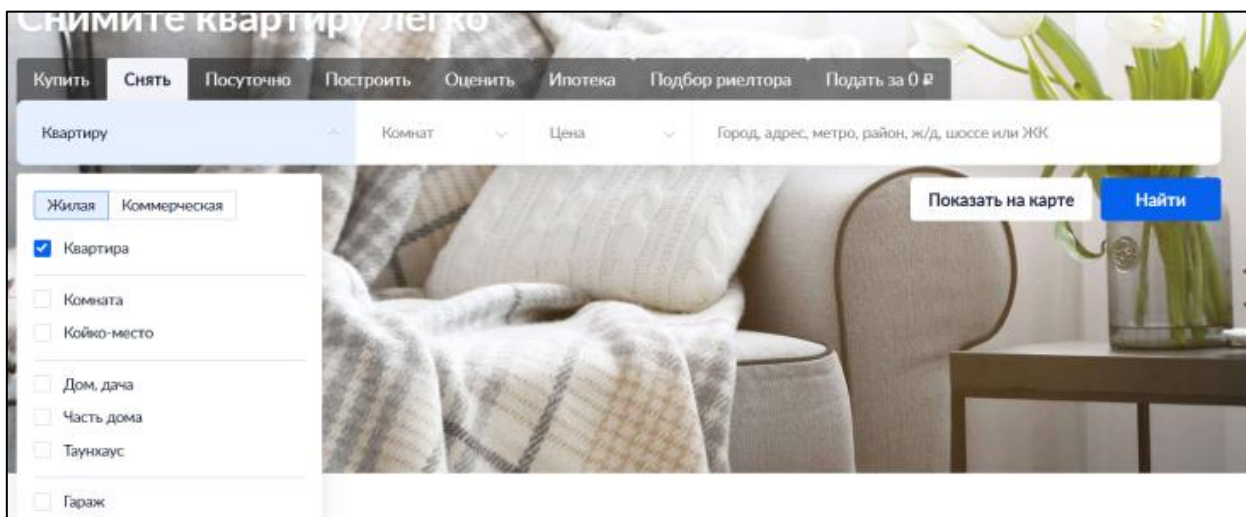


Рисунок 1.10 – Стартовый интерфейс подбора объектов на сайте «Циан»

Отображение результатов поиска. Каталог объектов представлен в виде вертикальной ленты карточек (рисунок 1.11). Вся самая важная информация сконцентрирована в одном компактном блоке, что избавляет пользователя от необходимости открывать множество вкладок. Основным преимуществом такого отображения является вынос ключевых параметров (цена, метраж, этаж, удаленность от метро) непосредственно в превью объявления. Это позволяет пользователю проводить быстрый сравнительный анализ объектов по техническим характеристикам, не переходя на страницу каждого лота. Такой подход делает взаимодействие с каталогом интуитивно понятным и не трудоемким.

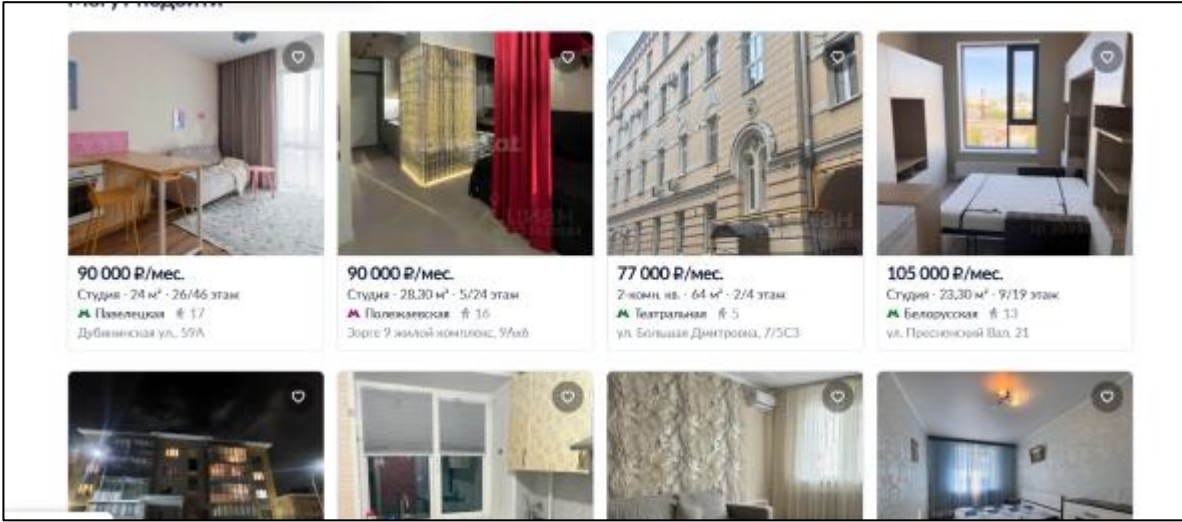


Рисунок 1.11 – Список объектов недвижимости в каталоге

Наполнение карточки объекта и инструменты связи. Детальная карточка объекта (рисунок 1.12) отличается глубокой технической проработкой. Информация разделена на логические блоки: «О квартире» и «О доме».

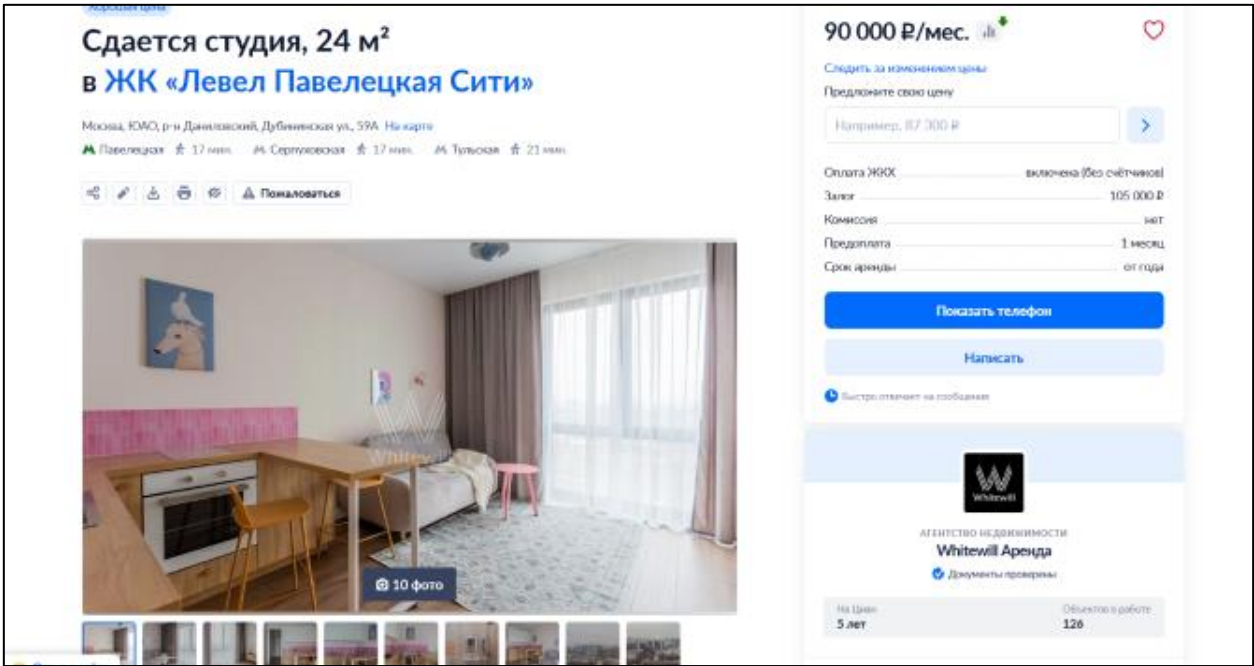


Рисунок 1.12 – Карточка объекта с инструментами связи и анализа цены

В системе отображаются данные о годе постройки, типе перекрытий, наличии парковки и лифтов. Также предусмотрены иконки удобств (кондиционер, мебель), что упрощает визуальное считывание информации (рисунок 1.13). Важным функциональным решением является блок взаимодействия. Помимо возможности просмотра номера телефона, в системе реализована функция «Написать» (чат), позволяющая клиенту вступить в прямую переписку с риелтором для уточнения деталей без совершения звонка. Также в карточке представлен уникальный инструмент «Предложите свою цену», стимулирующий начало переговоров.

О квартире

Подписаться на дом

Общая площадь	24 м²
Жилая площадь	15 м²
Площадь кухни	4 м²
Санузел	1 совмещенный
Вид из окон	На улицу
Ремонт	Дизайнерский

О доме

Подписаться на дом

Год постройки	2023
Строительная серия	Индивидуальный проект
Количество лифтов	2 пассажирских
Тип перекрытий	Нет информации
Подъезды	5
Парковка	Подземная
Отопление	Нет информации
Аварийность	Нет

В квартире есть

❄ Кондиционер

🛋 Мебель на кухне

🚿 Душевая кабина

🛏 Мебель в комнатах

Рисунок 1.13 – Детальные технические характеристики и удобства объекта

1.2 Постановка задачи

Обзор вышеперечисленных известных аналогов позволяет проанализировать все преимущества и недостатки существующих решений и сформулировать список требований, предъявляемых к программному средству, разрабатываемому в данном курсовом проекте. Программное средство должно обеспечивать эффективное взаимодействие между тремя категориями пользователей: клиентами, риелторами и администратором сервиса.

Разрабатываемое программное средство должно обеспечивать возможность выполнения перечисленных ниже функций:

- регистрация и авторизация: система регистрации и авторизации для разграничения прав доступа и управления личными данными пользователей в зависимости от их роли в системе;
- технический поиск и фильтрация: удобная система просмотра каталога недвижимости с использованием гибких фильтров (по цене, площади, типу объекта и локации) для быстрого подбора подходящих вариантов;
- управление избранным: функционал добавления интересующих объектов недвижимости в персональный список «Избранное» для их последующего сохранения и быстрого доступа к понравившимся вариантам;
- оформление заявок на показ: возможность для клиента подать онлайн-заявку на осмотр выбранного объекта в удобное время;
- личный кабинет риелтора: специализированный интерфейс для управления базой данных недвижимости (добавление, редактирование

описаний, загрузка фотографий) и обработки заявок клиентов со сменой их статусов;

- планирование и статистика: инструменты для создания расписания показов объектов риелторами и мониторинга личной эффективности сотрудников;

- административная панель: функционал для управления штатом риелторов, просмотра сводной статистики по агентству и управления базой данных объектов;

- модерация и отзывы: система публикации отзывов клиентами о работе агентства или конкретного риелтора с механизмом предварительной модерации администратором;

- система оповещений: автоматическое уведомление клиентов о статусе их заявок и новостях агентства посредством электронной почты.

В ходе анализа предметной области рассмотрены три существующих аналога: «Яндекс Недвижимость», «Домклик» и «Циан». Выявлены их ключевые особенности в части фильтрации объектов, организации каталога, инструментов обратной связи и личных кабинетов пользователей. На основе проведённого сравнительного анализа сформулирован перечень функциональных требований к разрабатываемому программному средству.

2 Анализ требований к программному средству и разработка функциональных требований

2.1 Описание средств разработки

При разработке программного средства для автоматизации работы агентства недвижимости были использованы следующие технологии и инструменты:

- интегрированная среда разработки JetBrains Rider;
- платформа .NET 8.0;
- язык программирования C#;
- декларативный язык разметки XAML;
- технология WPF (Windows Presentation Foundation);
- архитектурный паттерн MVVM;
- Entity Framework Core (версия 8.0.0);
- система управления базами данных Microsoft SQL Server.
- JetBrains Rider – интегрированная среда разработки с полной

поддержкой платформы .NET и технологии WPF. В процессе работы активно использовались встроенный визуальный редактор XAML, система отладки с точками останова и мощный инструмент JetBrains Backend для ускорения написания программного кода и его отладки.

.NET 8.0 – кроссплатформенная среда выполнения с открытым исходным кодом, выбранная за высокую производительность и расширенную поддержку современных библиотек. Данная версия платформы обеспечивает стабильную работу настольного приложения и эффективное управление ресурсами при работе с большими массивами данных.

C# – объектно-ориентированный язык программирования со строгой типизацией. В рамках проекта возможности языка позволили реализовать сложную бизнес-логику взаимодействия клиентов и риелторов. Особое внимание было уделено использованию технологии LINQ для быстрой фильтрации каталога недвижимости по множеству параметров, а также асинхронному программированию (async/await), что позволило избежать блокировок интерфейса при выполнении «тяжелых» запросов к базе данных или загрузке медиафайлов (фотографий объектов).

XAML (eXtensible Application Markup Language) – язык разметки, на котором описан внешний вид окон и элементов управления ПС. Использование XAML позволило гибко настроить визуальное представление каталога недвижимости, отделив структуру интерфейса от логики обработки событий.

WPF (Windows Presentation Foundation) – графическая подсистема для построения настольных приложений Windows. Технология позволила создать интерактивный интерфейс с использованием стилей и шаблонов данных, что необходимо для качественного отображения детальной информации об объектах недвижимости.

Entity Framework Core 8.0.0 – современный объектно-реляционный маппер (ORM). В проекте реализован подход Code First, при котором база данных (включая таблицы риелторов, объектов и отзывов) создавалась на

основе написанных классов C#. Применение EF Core позволило автоматизировать взаимодействие с данными, исключив написание сложных SQL-запросов вручную и обеспечив удобное управление структурой БД через механизм миграций.

Microsoft SQL Server – надежная реляционная система управления базами данных. Она используется в проекте в качестве централизованного хранилища всей информации агентства: от технических характеристик объектов недвижимости и фотографий до графиков показов и архивов клиентских заявок.

CommunityToolkit.Mvvm (версия 8.2.2) – библиотека для упрощённой реализации паттерна MVVM. Все классы ViewModel наследуются от базового класса ObservableObject. Атрибут [ObservableProperty] автоматически генерирует реактивные свойства с уведомлением об изменении через INotifyPropertyChanged, а [RelayCommand] – реализации ICommand с поддержкой CanExecute без написания шаблонного кода вручную.

BCrypt.Net-Next (версия 4.0.3) – библиотека для хэширования паролей по алгоритму bcrypt. Применяется при регистрации пользователя (BCrypt.HashPassword) и верификации пароля при входе в систему (BCrypt.Verify). Алгоритм bcrypt является адаптивным и устойчивым к атакам перебором, что исключает хранение паролей в открытом виде в базе данных.

LiveChartsCore.SkiaSharpView.WPF (версия 2.0.0) – библиотека для построения интерактивных графиков в WPF-приложениях. Используется на вкладках «Статистика» в панелях риелтора и администратора: линейный график динамики заявок по месяцам и столбчатая диаграмма продаж по агентству.

Microsoft.Extensions.DependencyInjection (версия 8.0.0) – контейнер инверсии зависимостей. Все сервисы, репозитории и классы ViewModel зарегистрированы в точке входа приложения и разрешаются через IServiceProvider, что обеспечивает слабую связанность компонентов системы и возможность их независимого замещения.

Архитектурный паттерн MVVM (Model-View-ViewModel) – фундаментальный подход к проектированию WPF-приложений. Паттерн обеспечил разделение системы на три независимых слоя:

- Model – содержит сущности базы данных (Объект, Риелтор, Заявка) и логику работы с ними. Сюда выносятся всё, что связано с непосредственным получением, хранением и валидацией данных. Model ничего не знает о том, как эти данные будут показаны пользователю, и не содержит ссылок на View.

- View – графический интерфейс, с которым взаимодействует пользователь. Задача View сводится исключительно к отрисовке элементов и передаче пользовательских действий (нажатий, ввода текста) во ViewModel.

- ViewModel – логика представления, связывающая данные из модели с их отображением на экране без прямой зависимости слоев друг от друга, что значительно упрощает тестирование и дальнейшее развитие функционала агентства.

2.2 Описание разрабатываемой функциональности программного средства

На основе проведенного анализа аналогов и требований, указанных в постановке задачи, была определена функциональность разрабатываемого программного средства «Агентство недвижимости». В данном разделе описаны основные функции системы, пользовательские роли и требования к взаимодействию различных категорий пользователей с программным продуктом.

2.2.1 Пользовательские роли в системе

В программном средстве «Агентство недвижимости» выделены три основные пользовательские роли, для каждой из которых предусмотрен уникальный набор инструментов и прав доступа:

Администратор системы – пользователь с наиболее широкими правами доступа, осуществляющий глобальный контроль за работой всей платформы. Администратор отвечает за управление ролями пользователей (повышение клиентов до роли риелтора и обратное понижение, блокировка учетных записей), модерацию поступающих отзывов для обеспечения чистоты контента, а также за массовое оповещение клиентской базы по электронной почте. В зону ответственности администратора также входит ведение базы данных объектов недвижимости и анализ сводной статистики по эффективности работы каждого риелтора.

Риелтор – сотрудник агентства, основной задачей которого является работа с базой объектов и обработка заявок. Риелтор обладает правами на добавление новых объектов недвижимости, редактирование текущих описаний и загрузку фотографий. Программное средство позволяет риелтору управлять расписанием показов, изменять статусы заявок клиентов в процессе работы и отслеживать показатели личной статистики.

Клиент – конечный потребитель услуг агентства. Зарегистрированный пользователь данной роли имеет возможность авторизоваться в системе для управления личными данными. Клиенту доступен просмотр актуального каталога недвижимости с применением сложной системы фильтрации. Система позволяет клиенту добавлять понравившиеся объекты в раздел «Избранное» для последующего быстрого доступа, оформлять онлайн-заявки на показ выбранных объектов, а также публиковать отзывы о качестве работы агентства или конкретного риелтора.

Указанные роли определяют строгий перечень доступных функций для каждого типа пользователя, обеспечивают надежное разграничение прав доступа и способствуют структурированной автоматизации бизнес-процессов агентства недвижимости. Администратор управляет кадрами и контентом, не затрагивая операционную работу риелторов. Риелтор сосредоточен на объектах и заявках, не имея доступа к глобальным настройкам. Клиент же

работает в безопасной среде, где его данные изолированы, а отзывы проверены администратором.

2.2.2 Функциональные требования

2.2.2.1 Общие функции системы

Регистрация и авторизация пользователей:

- регистрация новых пользователей в системе с указанием персональных и контактных данных;
- авторизация зарегистрированных пользователей и разграничение прав доступа согласно ролям (администратор, риелтор, клиент).

Каталог объектов недвижимости:

- просмотр актуального списка объектов недвижимости;
- использование системы фильтров для точного подбора (по цене, площади, количеству комнат и типу объекта);
- детальный просмотр информации об объекте, включая технические характеристики и фотографии.

Система оповещений:

- автоматическая отправка уведомлений пользователям по электронной почте при изменении статусов заявок или публикации новостей агентства.

2.2.2.2 Функции администратора

Управление пользователями. Администратор имеет возможность просматривать список всех зарегистрированных пользователей, повышать роль клиента до риелтора, понижать риелтора обратно до клиента, а также блокировать и разблокировать учетные записи при необходимости.

Модерация отзывов. Проверка поступающих от клиентов отзывов о работе агентства или риелторов перед их публикацией на сайте для обеспечения достоверности данных.

Оповещение клиентов. Функционал для создания и рассылки информационных сообщений по базе пользователей через электронную почту.

Управление базой данных объектов. Глобальный контроль над каталогом недвижимости, возможность внесения корректировок в любые записи базы данных.

Просмотр статистики. Доступ к аналитическим отчетам по эффективности работы каждого риелтора (количество сделок, средний рейтинг, активность).

2.2.2.3 Функции риелтора

Управление объектами недвижимости. Возможность добавления новых предложений в каталог, редактирование описаний существующих объектов и загрузка фотографий.

Обработка заявок клиентов. Прием входящих запросов на просмотр объектов и оперативная смена их статусов в процессе работы с клиентом. Заявки отображаются в порядке поступления для соблюдения очередности обработки. Смена статуса сразу видна клиенту в личном кабинете.

Создание расписания показов. Настройка личного графика работы с указанием свободных временных слотов для проведения встреч и осмотров недвижимости.

Просмотр личной статистики. Мониторинг собственных показателей продуктивности и ознакомление с оценками, выставленными клиентами.

2.2.2.4 Функции клиента

Управление личными данными. Возможность регистрации, авторизации и редактирования информации в персональном профиле.

Просмотр каталога. Использование расширенного поиска и фильтрации для подбора оптимальных вариантов жилья из общей базы агентства.

Оформление заявки на показ. Возможность выбора объекта и отправки запроса на его осмотр в удобное время согласно графику риелтора.

Работа с «Избранным». Добавление понравившихся квартир или домов в личный список для последующего быстрого доступа к сохранённым объектам.

Публикация отзывов. Возможность оставить развернутый отзыв и выставить оценку риелтору или агентству после завершения взаимодействия.

2.2.3 Диаграмма вариантов использования

Для наглядного представления функциональных требований была разработана диаграмма вариантов использования, отражающая взаимодействие пользователей (клиента, риелтора и администратора) с системой. Данная диаграмма вариантов использования представлена в приложении А.

2.2.4 Спецификация функциональных требований

2.2.4.1 Требования к регистрации и авторизации

Система должна предоставлять форму для регистрации клиента, которая включает поля: логин, полное имя, адрес электронной почты, номер мобильного телефона (необязательно), пароль и подтверждение пароля. Пароль должен быть длиной не менее 6 символов. После заполнения формы система должна выполнить проверку уникальности логина и адреса электронной почты в базе данных.

Для авторизации пользователей всех ролей система предоставляет форму с полями для логина и пароля. При вводе неверных данных система выводит информационное сообщение об ошибке. Успешная проверка учетных данных, включающая сравнение хэшей паролей по алгоритму BCrypt и

проверку статуса блокировки аккаунта, перенаправляет пользователя на главную страницу, соответствующую его роли: каталог для клиента, панель управления для риелтора или администратора.

2.2.4.2 Требования к работе с объектами недвижимости

Для добавления объекта недвижимости риелтор должен иметь доступ к форме, включающей поля: название объекта, адрес, тип недвижимости (квартира, дом, коммерческая), общая площадь, количество комнат, этаж, техническое описание и стоимость. Необходимо реализовать возможность загрузки нескольких фотографий объекта. Система должна поддерживать автоматическое масштабирование изображений для корректного отображения.

Пользователи должны иметь возможность просматривать актуальный список доступной недвижимости с применением фильтров. При выборе конкретного объекта должна открываться страница с его детальными характеристиками, фотогалереей и графиком доступных для показа временных слотов. Клиент должен иметь возможность добавлять объекты в раздел «Избранное».

2.2.4.3 Требования к уведомлениям и отзывам

Система поддерживает автоматическую отправку email-уведомлений пользователям в следующих случаях: при регистрации нового аккаунта (приветственное письмо), при изменении статуса заявки на показ (подтверждение или отмена), а также при добавлении в каталог каждого десятого активного объекта (оповещение всех клиентов о пополнении базы). Уведомления отправляются через SMTP-сервер, настроенный в конфигурационном файле приложения.

Для реализации обратной связи система должна предоставлять клиенту форму публикации отзыва. Форма включает текстовое поле для комментария и возможность выбора оценки по 5-балльной шкале. Отзывы должны отображаться на странице риелтора или агентства после прохождения модерации.

2.2.4.4 Требования к администрированию и отчетности

Для управления пользователями администратор должен иметь интерфейс со списком всех зарегистрированных аккаунтов с возможностью фильтрации по роли. Администратор может изменять роль пользователя (клиент – риелтор), а также блокировать и разблокировать учетные записи.

В разделе модерации администратор должен иметь интерфейс для проверки отзывов, где он может одобрять их для публикации или удалять.

В разделе управления заявками риелтор и администратор должны иметь возможность просматривать список входящих запросов на показы с

фильтрацией по статусу («Новая», «Подтверждена», «Завершена», «Отменена») и иметь инструменты для оперативной смены этих статусов.

Система должна предоставлять интерфейс для просмотра статистики: для администратора – сводная по всему агентству, для риелтора – персональная статистика по количеству сделок и среднему рейтингу.

В данном разделе описан выбранный технологический стек и обоснована целесообразность применения каждого инструмента. Определены три пользовательские роли системы (клиент, риелтор, администратор) и разработаны функциональные требования к каждой из них. Сформулированы спецификации к регистрации и авторизации, работе с объектами недвижимости, системе уведомлений и административному управлению. Совокупность требований является основой для проектирования архитектуры программного средства.

3 Проектирование программного средства

3.1 Архитектура системы

Программное средство «Агентство недвижимости» разработано с использованием архитектурного паттерна MVVM (Model-View-ViewModel), который обеспечивает четкое разделение ответственностей между компонентами приложения, что способствует улучшению структуры кода, упрощает тестирование и обеспечивает легкую поддержку.

Архитектура MVVM состоит из трех основных компонентов:

- Model (Модель) – содержит классы-сущности базы данных (User, Property, Appointment, Review и др.) и логику работы с ними через сервисный слой;
- View (Представление) – XAML-файлы, описывающие визуальное представление страниц: каталога объектов, личного кабинета клиента, панелей риелтора и администратора;
- ViewModel (Модель представления) – выступает в качестве посредника между представлением и моделью, обрабатывает команды пользователя и предоставляет данные для отображения.

Структура шаблона MVVM представлена на рисунке 3.1.

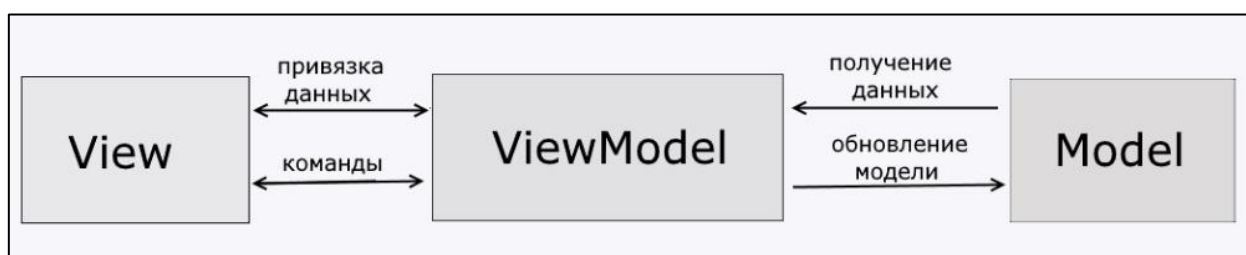


Рисунок 3.1 – Структура шаблона MVVM

Помимо основного паттерна MVVM, в архитектуре приложения применяются следующие паттерны проектирования:

- репозиторий (Repository) – обеспечивает абстракцию доступа к данным через интерфейс IUnitOfWork, скрывая детали работы с Entity Framework Core;
- сервисный слой (Service Layer) – бизнес-логика вынесена в отдельные сервисы (IAuthService, IPropertyService, IAppointmentService и др.), что обеспечивает независимость логики от слоя представления;
- команда (Command) – инкапсуляция действий пользовательского интерфейса реализована через механизм RelayCommand библиотеки CommunityToolkit.Mvvm;
- наблюдатель (Observer) – реализован через механизм привязки данных WPF и атрибут [ObservableProperty] библиотеки CommunityToolkit.Mvvm, обеспечивает автоматическое обновление интерфейса при изменении данных;
- внедрение зависимостей (Dependency Injection) – все компоненты системы регистрируются в контейнере Microsoft.Extensions.DependencyInjection с различными жизненными циклами:

Singleton – для NavigationService, HeaderViewModel и сервиса уведомлений; Scoped – для всех ViewModel и сервисов бизнес-логики. При каждом переходе между страницами; NavigationService создаёт новый DI-скоуп, что гарантирует актуальное состояние данных и автоматическое освобождение ресурсов.

3.2 Диаграмма компонентов

Для отображения физической структуры программного средства и взаимосвязей между его модулями разработана диаграмма компонентов, представленная в приложении Б.

Диаграмма компонентов является статической структурной диаграммой UML и отображает разбиение системы на структурные компоненты и зависимости между ними. В разработанной системе выделены следующие компоненты: исполняемый модуль приложения (MSM.exe), модуль представления (Views), модуль логики представления (ViewModels), модуль сервисного слоя (Services), модуль доступа к данным (Data), модуль моделей предметной области (Models), а также внешние зависимости – библиотека Entity Framework Core и сервер баз данных Microsoft SQL Server.

Зависимости между компонентами демонстрируют направление потоков данных: модуль Views зависит от ViewModels, ViewModels зависят от сервисного слоя, сервисный слой взаимодействует с репозиториями через интерфейс IUnitOfWork, репозитории работают с базой данных через AppDbContext.

3.3 Взаимоотношения между классами

Для визуализации структуры системы и отображения взаимосвязей между её основными компонентами была построена диаграмма классов, представленная в Приложении В.

Диаграмма классов является одним из ключевых элементов UML и служит для описания классов, их свойств, методов, а также связей между ними – таких как ассоциации, наследование, агрегация и композиция.

Созданная диаграмма отражает архитектуру приложения, включая основные сущности: пользователь (User), объект недвижимости (Property), заявка на показ (Appointment), отзыв (Review), избранное (Favorite) и расписание риелтора (RealtorSchedule). Диаграмма демонстрирует применённые принципы объектно-ориентированного проектирования и помогает лучше понять структуру системы, что облегчает её сопровождение и возможное расширение в будущем.

3.4 Проектирование модели базы данных

Для хранения данных программного средства «Агентство недвижимости» разработана реляционная модель базы данных, которая позволяет эффективно организовать информацию о пользователях, объектах

недвижимости, заявках на показ, отзывах и избранном. Логическая модель базы данных, отражающая таблицы, их атрибуты и связи между ними, представлена на рисунке 3.2.

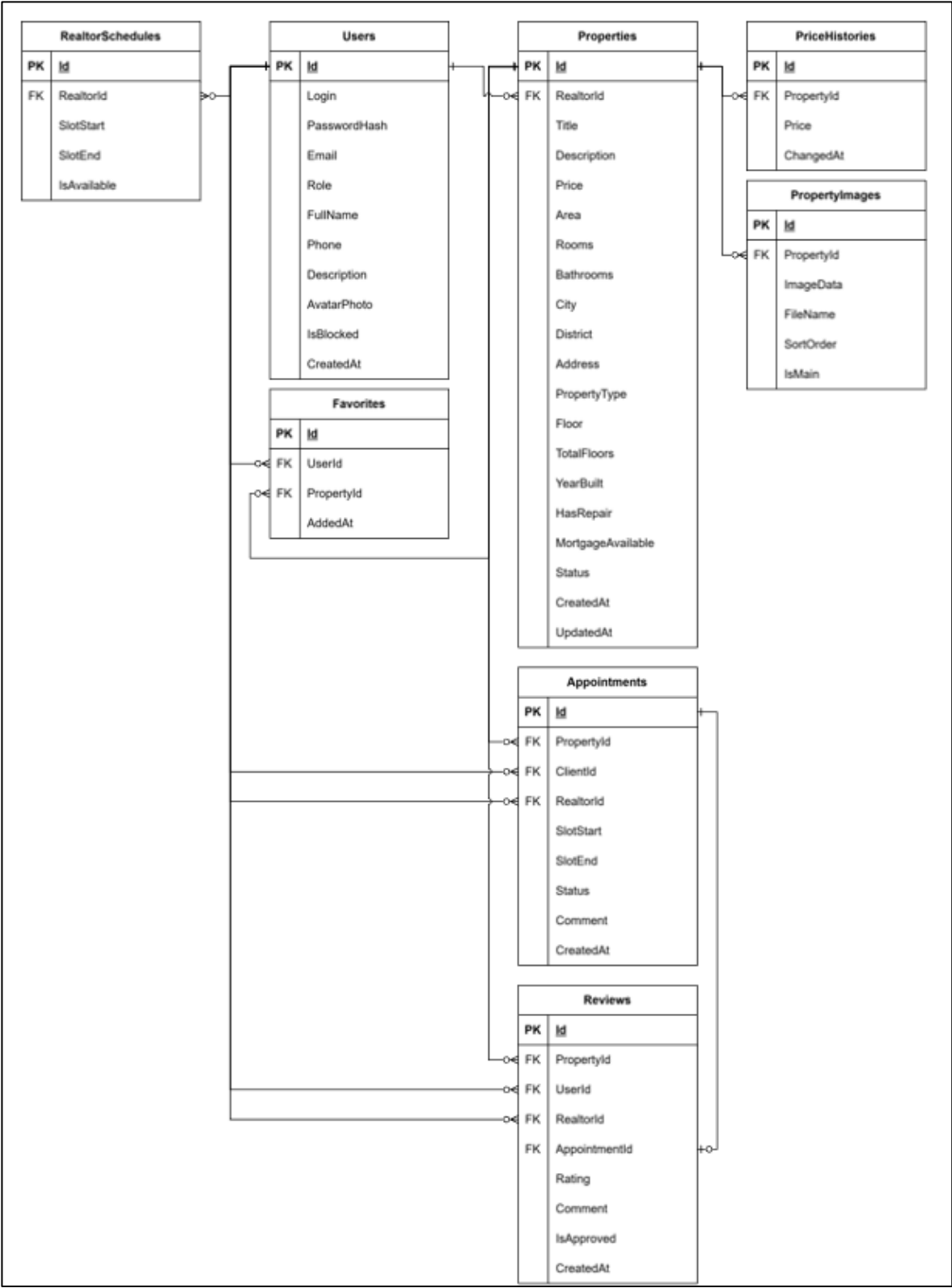


Рисунок 3.2 – Логическая модель базы данных

Разработанная модель базы данных охватывает основные сущности программного средства «Агентство недвижимости». Центральной таблицей

является Users, хранящая информацию о пользователях всех ролей (администратор, риелтор, клиент). С ней связаны таблицы Properties (объекты недвижимости), Appointments (заявки на показ), Reviews (отзывы) и Favorites (избранное). Таблица PropertyImages хранит фотографии объектов, PriceHistory – историю изменения цен, RealtorSchedules – заблокированные интервалы в расписании риелторов. Реляционный подход обеспечивает структурированное хранение данных. Ограничения целостности (первичные и внешние ключи, значения по умолчанию) способствуют сохранению согласованности данных и корректной работе системы.

3.5 Проектирование последовательности взаимодействия

В процессе проектирования приложения для отображения взаимодействия между его объектами была использована диаграмма последовательности, являющаяся одним из основных инструментов языка UML.

Этот тип диаграмм позволяет отразить порядок обмена сообщениями между участниками процесса и широко применяется для описания логики работы системы, взаимодействия между её компонентами, пользовательских сценариев и процессов обработки данных.

Диаграмма последовательности, демонстрирующая взаимодействие пользователя с системой в процессе авторизации, просмотра каталога объектов недвижимости и оформления заявки на показ, приведена в Приложении Г.

В данном разделе спроектирована архитектура программного средства на основе паттерна MVVM с применением паттернов Repository, Unit of Work, Service Layer, Command, Observer и Dependency Injection. Построены диаграмма компонентов, диаграмма классов, логическая модель базы данных и диаграмма последовательности. Разработанная реляционная модель включает восемь взаимосвязанных таблиц и охватывает все объекты предметной области агентства недвижимости.

4 Реализация программного средства

4.1 Реализация архитектуры проекта на основе MVVM

Программное средство «Агентство недвижимости» реализовано в соответствии с архитектурным паттерном MVVM, который обеспечивает четкое разделение ответственности между компонентами приложения, описание паттерна представлено в главе 3.1. Структура проекта показана на рисунке 4.1.

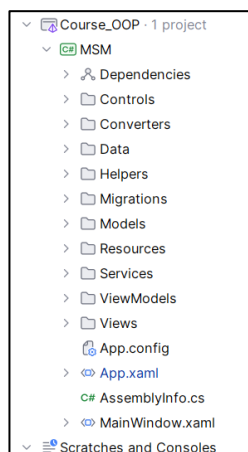


Рисунок 4.1 – Структура проекта

Программное средство структурировано в виде отдельных модулей, каждый из которых выполняет определённую роль в системе:

- Controls – содержит переиспользуемые пользовательские элементы управления (карточка объекта недвижимости, карусель изображений);
- Converters – содержит классы преобразования данных для корректного отображения в интерфейсе (конвертеры цены, булевых значений, изображений);
- Data – модуль для работы с данными, включающий классы контекста базы данных (AppDbContext), репозитории и паттерна Unit of Work;
- Migrations – содержит файлы миграций базы данных, обеспечивающие управление версионированием схемы БД и возможность обновления структуры данных;
- Models – классы, описывающие основные сущности базы данных (User, Property, Appointment, Review и др.);
- Resources – содержит статические ресурсы проекта: изображения, темы оформления (светлая и тёмная), словари локализации (русский и английский);
- Services – классы сервисного слоя, реализующие бизнес-логику системы (IAuthService, IPropertyService, IAppointmentService, INotificationService и др.);
- ViewModels – содержит классы, реализующие логику представления и обработку взаимодействия с пользователем;

– Views – XAML-файлы, реализующие визуальные компоненты интерфейса приложения.

Конфигурация всех компонентов системы выполняется в методе `ConfigureServices` класса `App.xaml.cs` с использованием контейнера `Microsoft.Extensions.DependencyInjection`. Сервисы `NavigationService`, `HeaderViewModel` и `EmailNotificationService` зарегистрированы как `Singleton` – создаются один раз и существуют на протяжении всего сеанса. Сервисы бизнес-логики и все классы `ViewModel` зарегистрированы как `Scoped` – создаются заново при каждом переходе между страницами. При вызове `NavigateTo()` сервис навигации создаёт новый DI-скоуп и разрешает из него требуемый `ViewModel`, что гарантирует актуальное состояние данных на каждой странице и автоматическое освобождение ресурсов.

4.2 Реализация ключевых функций

В данном разделе представлены ключевые функции программного средства «Агентство недвижимости», реализованные в соответствии с требованиями, изложенными в предыдущих главах. Каждая функция разработана с учетом архитектурного паттерна MVVM.

4.2.1 Реализация функции авторизации и регистрации

Функции авторизации и регистрации являются базовыми для программного средства, поскольку обеспечивают доступ к системе и определяют права пользователя в зависимости от его роли.

Процесс регистрации включает в себя следующие этапы:

- ввод регистрационных данных (логин, полное имя, электронная почта, пароль, подтверждение пароля);
- валидация введенных данных (проверка заполненности обязательных полей, минимальной длины пароля – не менее 6 символов);
- проверка уникальности логина и адреса электронной почты в базе данных;
- хэширование пароля с использованием алгоритма `BCrypt`;
- сохранение данных пользователя в базе данных;
- автоматическая отправка приветственного email-уведомления.

Процесс авторизации включает следующие шаги:

- ввод пользователем логина и пароля;
- поиск пользователя в базе данных по логину;
- верификация пароля путём сравнения хэшей с использованием `BCrypt`;
- проверка статуса блокировки учётной записи;
- перенаправление на страницу согласно роли пользователя (клиент – каталог, риелтор – панель риелтора, администратор – панель администратора).

Реализация функции регистрации представлена в приложении Е, функция авторизации – в приложении Ж. Блок-схема алгоритма регистрации пользователя представлена в приложении Д.

4.2.2 Реализация функции добавления объекта недвижимости

Функция добавления объекта недвижимости позволяет пользователям с ролью «Риелтор» размещать новые предложения в каталоге агентства. Процесс добавления реализован в несколько этапов:

- заполнение формы с информацией об объекте (название, адрес, тип, площадь, количество комнат, этаж, описание, стоимость);
- загрузка одной или нескольких фотографий объекта с возможностью указания главного изображения;
- валидация введенных данных;
- сохранение объекта и связанных изображений в базе данных;
- автоматическое email-уведомление всем клиентам при добавлении каждого десятого активного объекта в каталог.

Реализация функции добавления объекта недвижимости представлена в приложении 3.

4.2.3 Реализация функции оформления заявки на показ

Функция оформления заявки на показ объекта является одной из ключевых для системы агентства недвижимости и реализована следующим образом:

- выбор объекта недвижимости из каталога;
- переход на страницу детального просмотра объекта;
- выбор доступного временного слота из расписания риелтора;
- ввод комментария к заявке (необязательно);
- сохранение заявки в базе данных со статусом «Новая»;
- автоматическое email-уведомление клиенту при изменении статуса заявки риелтором (подтверждение или отмена).

Реализация функции оформления заявки на показ представлена в приложении И.

4.2.4 Реализация системы многокритериальной фильтрации каталога

Подбор объектов по параметрам реализован в методе `GetFilteredAsync` класса `PropertyService` с применением технологии `LINQ to Entities`. Для отделения логики построения запроса от его выполнения выделен вспомогательный приватный метод `ApplyFilters`, принимающий объект `IQueryable` и набор параметров. Каждый из девяти параметров применяется к запросу только при наличии значения, что позволяет строить запросы произвольной сложности без дублирования кода.

Поддерживаются следующие критерии фильтрации: диапазон цены и площади, количество комнат и санузлов, город, район, тип объекта, наличие ипотеки и ремонта, а также текстовый поиск по названию и описанию. Сортировка реализована через конструкцию `switch` с четырьмя режимами: по

цене и площади в обоих направлениях. При отсутствии явного параметра сортировки применяется упорядочение по дате добавления.

Для оптимизации производительности реализовано страничное отображение: параметры `skip` и `take` передаются напрямую в запрос к базе данных. В `PropertyListViewModel` предусмотрена функция «Показать ещё», подгружающая следующие двенадцать объектов без перезагрузки всего списка, что снижает нагрузку на базу данных и сокращает время ожидания пользователя.

4.2.5 Реализация системы email-уведомлений

Автоматические уведомления реализованы в классе `EmailNotificationService`, реализующем интерфейс `INotificationService`. Сервис зарегистрирован как `Singleton`, что дает единый экземпляр на протяжении всего сеанса и исключает повторную инициализацию конфигурации SMTP.

Конфигурация SMTP-сервера вынесена в файл `App.config` в виде именованных ключей (`Smtp:Host`, `Smtp:Port`, `Smtp:UserName`, `Smtp:Password`). При запуске сервис проверяет корректность конфигурации: если поля не заполнены или содержат значения-заглушки, отправка писем отключается без выброса исключений, что не нарушает работу приложения при отсутствии SMTP-сервера.

Реализованы четыре шаблона уведомлений:

- приветственное письмо при регистрации нового пользователя;
- уведомление клиенту об изменении статуса заявки риелтором (подтверждена, отменена, завершена) с указанием объекта, имени риелтора и времени показа;
- массовое оповещение всех клиентов при достижении кратного десяти числа активных объектов в каталоге;
- произвольная рассылка от имени администратора с выбором группы получателей.

Массовые рассылки выполняются через `Task.WhenAll`, обеспечивая параллельную отправку всем получателям. Уведомления о статусе заявки вызываются в режиме `fire-and-forget` для исключения блокировки пользовательского интерфейса при выполнении сетевых операций.

4.2.6 Реализация паттернов Repository и Unit of Work

Доступ к данным организован через паттерн `Repository` совместно с `Unit of Work`. Обобщённый интерфейс `IRepository` определяет единый контракт для работы с любой сущностью: получение по идентификатору, поиск по предикату `Expression<Func<T, bool>>`, проверку существования, подсчёт, добавление и удаление. Класс `Repository` реализует этот интерфейс, делегируя вызовы соответствующему `DbSet` контекста `Entity Framework Core`.

Интерфейс `IUnitOfWork` агрегирует восемь типизированных репозиториях (по одному на каждую сущность: `Users`, `Properties`,

PropertyImages, Appointments, RealtorSchedules, Favorites, Reviews, PriceHistories) и предоставляет методы управления транзакциями: BeginTransactionAsync, CommitTransactionAsync, RollbackTransactionAsync и SaveChangesAsync. Класс UnitOfWork использует отложенную инициализацию репозитория через оператор `??=`, что исключает создание лишних объектов.

В методах с высокими требованиями к согласованности данных применяются транзакции с уровнем изоляции `Serializable`. В `BookSlotAsync` это исключает двойное бронирование одного временного слота при одновременных запросах нескольких клиентов. В `ReviewService.CreateAsync` – предотвращает создание двух отзывов по одной завершённой записи.

4.3 Реализация модели данных

Модель данных в системе агентства недвижимости представляет собой ключевую часть архитектуры, отвечающую за хранение и управление всей информацией, связанной с объектами недвижимости, пользователями, заявками и другими сущностями. В системе используется объектно-реляционное отображение.

Данная модель реализована с использованием Entity Framework Core по принципу Code First, что позволило описать структуру базы данных через классы C#. Структура базы данных была подробно описана в разделе 3.4. Пример подхода Code First представлен в листинге 4.1.

```
namespace MSM.Models.Entities;
public class User : BaseEntity
{
    public string Login { get; set; } = string.Empty;
    public string PasswordHash { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public string Role { get; set; } = string.Empty;
    public string FullName { get; set; } = string.Empty;
    public string? Phone { get; set; }
    public string? Description { get; set; }
    public byte[]? AvatarPhoto { get; set; }
    public bool IsBlocked { get; set; } = false;
    public DateTime CreatedAt { get; set; } = DateTime.Now;
    public ICollection<Property>? Properties { get; set; }
    public ICollection<Appointment>? ClientAppointments { get;
set; }
    public ICollection<Appointment>? RealtorAppointments {
get; set; }
    public ICollection<Favorite>? Favorites { get; set; }
    public ICollection<Review>? Reviews { get; set; }
    public ICollection<RealtorSchedule>? Schedules { get; set;
}}
}
```

Листинг 4.1 – Реализация модели данных пользователя

Также для реализации принципа Code First был создан класс `AppDbContext`, который наследуется от `DbContext`. Этот класс служит для описания структуры базы данных и управления взаимодействием с ней. Он определяет таблицы (`DbSet`), связи между сущностями и настройки для каждой модели. Через `AppDbContext` осуществляется выполнение операций с базой данных, таких как добавление, изменение, удаление и чтение данных. Класс также используется для применения миграций, что позволяет автоматически обновлять структуру базы данных при изменении модели данных.

Реализация данного класса представлена в приложении К.

В данном разделе описана реализация программного средства в соответствии с спроектированной архитектурой MVVM. Рассмотрена структура проекта, конфигурация контейнера зависимостей и назначение каждого модуля. Подробно описаны шесть ключевых функций: регистрация и авторизация с хэшированием паролей `BCrypt`, добавление объекта недвижимости с загрузкой фотографий в транзакции, оформление заявки на показ с проверкой конфликтов слотов, многокритериальная фильтрация каталога с страничной загрузкой, система email-уведомлений, а также реализация паттернов `Repository` и `Unit of Work`. Реализация модели данных выполнена по принципу Code First с использованием `Entity Framework Core`.

5 Тестирование, проверка работоспособности и анализ полученных результатов

5.1 Тестирование сервиса регистрации и авторизации

Сервис регистрации и авторизации является важным компонентом системы, обеспечивающим безопасность пользователей и контроль доступа к функционалу агентства недвижимости. Для проверки работоспособности данного сервиса были разработаны тестовые сценарии (тест-кейсы), результаты выполнения которых представлены в таблицах 5.1 и 5.2.

Таблица 5.1 – Тестирование модуля регистрации пользователя

Шаг	Действие пользователя	Ожидаемый результат	Фактический результат	Статус
1	2	3	4	5
1	Ввод адреса электронной почты в некорректном формате (без «@») и нажатие кнопки «Зарегистрироваться»	Система отображает сообщение об ошибке формата email; регистрация не выполняется	Система выявила ошибку и вывела уведомление; переход к следующему шагу заблокирован	Успешно
2	Ввод пароля длиной менее 6 символов	Кнопка регистрации остаётся неактивной; отображается сообщение об ошибке	Кнопка заблокирована через CanExecute в RelayCommand; сообщение об ошибке выведено	Успешно
3	Попытка регистрации с логином, уже занятым другим пользователем	Система отображает предупреждение о занятом логине; учётная запись не создаётся	Проверка уникальности в БД выполнена; предупреждение отображено корректно	Успешно
4	Заполнение всех обязательных полей корректными данными и нажатие «Зарегистрироваться»	Учётная запись создана; отправлено приветственное email; переход на страницу авторизации	Пользователь зарегистрирован; email отправлен; выполнен редирект на страницу входа	Успешно

Таблица 5.2 – Тестирование модуля авторизации пользователя

Шаг	Действие пользователя	Ожидаемый результат	Фактический результат	Статус
1	2	3	4	5
1	Ввод верного логина и неверного пароля	Система отображает сообщение «Неверный логин или пароль»; вход не выполняется	Система сравнила хэши паролей через BCrypt; при несовпадении данных ошибка отображена корректно в интерфейсе.	Успешно

Продолжение таблицы 5.2 – Тестирование модуля авторизации пользователя

1	2	3	4	5
2	Ввод данных заблокированной учётной записи	Система отображает сообщение о блокировке; вход запрещён	Система проверила флаг IsBlocked и вывела сообщение о блокировке	Успешно
3	Ввод корректных данных пользователя с ролью «Клиент»	Выполнен вход; переход на страницу каталога объектов	Авторизация прошла успешно; выполнен переход на страницу каталога	Успешно
4	Ввод корректных данных пользователя с ролью «Риелтор»	Выполнен вход; переход на панель управления риелтора	Авторизация прошла успешно; выполнен переход на панель риелтора	Успешно

Выводы по разделу 5.1. В ходе тестирования модулей регистрации и авторизации проверены сценарии работы с некорректными данными, дублирующими учётными записями и заблокированными аккаунтами. Все тестовые сценарии завершились успешно, что подтверждает надёжность реализованного механизма аутентификации с хэшированием паролей по алгоритму BCrypt и разграничением доступа по ролям.

5.2 Тестирование функций риелтора

5.2.1 Тестирование функции добавления объекта недвижимости

При добавлении объекта недвижимости риелтор обязан заполнить все обязательные поля формы. Для проверки данной функции были разработаны тестовые сценарии, результаты которых представлены в таблице 5.3.

Таблица 5.3 – Тестирование функции добавления объекта недвижимости

Шаг	Действие пользователя	Ожидаемый результат	Фактический результат	Статус
1	2	3	4	5
1	Попытка сохранить объект с незаполненными обязательными полями (название, цена)	Кнопка сохранения остаётся неактивной; отображается сообщение об ошибке	Кнопка заблокирована; система отображает соответствующее предупреждение	Успешно
2	Заполнение всех полей, загрузка фотографии и нажатие «Сохранить»	Объект сохранён в БД; отображается в списке «Мои объекты»; создана запись в PriceHistory	Объект добавлен; фото сохранено; запись в PriceHistory создана атомарно в транзакции	Успешно
3	Добавление объекта при достижении кратного 10 числа активных объектов	Всем клиентам отправляется email-уведомление о пополнении каталога	Email-уведомление отправлено всем клиентам в режиме fire-and-forget	Успешно

Выводы по разделу 5.2.1. Тестирование функции добавления объекта недвижимости подтвердило корректную работу валидации формы, атомарное сохранение объекта, фотографий и истории цен в рамках транзакции, а также автоматическую отправку email-уведомлений клиентам при пополнении каталога.

5.2.2 Тестирование функции управления заявками

При обработке входящих заявок риелтор может изменять их статус. Тестовые сценарии проверки данной функции и их результаты представлены в таблице 5.4.

Таблица 5.4 – Тестирование функции управления заявками риелтора

Шаг	Действие пользователя	Ожидаемый результат	Фактический результат	Статус
1	2	3	4	5
1	Нажатие «Подтвердить» для заявки со статусом «Новая»	Статус меняется на «Подтверждена»; клиенту отправляется уведомление	Статус обновлён; email-уведомление отправлено клиенту автоматически	Успешно
2	Нажатие «Отменить» для заявки со статусом «Подтверждена»	Статус меняется на «Отменена»; клиенту отправляется уведомление	Статус обновлён; уведомление об отмене отправлено клиенту	Успешно
3	Нажатие «Завершить» для заявки со статусом «Подтверждена»	Статус меняется на «Завершена»; у клиента появляется кнопка отзыва	Статус обновлён; кнопка «Оставить отзыв» появилась у клиента	Успешно

Выводы по разделу 5.2.2. Тестирование функции управления заявками показало корректную смену статусов риелтором, автоматическое уведомление клиента по почте и соответствие переходов бизнес-логике.

5.3 Тестирование функций клиента

5.3.1 Тестирование функции оформления заявки на показ

При оформлении заявки клиент выбирает объект из каталога и переходит на его детальную страницу. Тестовые сценарии и их результаты представлены в таблице 5.5.

Таблица 5.5 – Тестирование функции оформления заявки на показ

Шаг	Действие пользователя	Ожидаемый результат	Фактический результат	Статус
1	2	3	4	
1	Открытие формы записи на показ на странице объекта	Отображается форма с выбором даты и временных слотов риелтора	Форма открылась; слоты 9:00–18:00 отображены; занятые выделены серым	Успешно

Продолжение таблицы 5.5 – Тестирование функции оформления заявки на показ

1	2	3	4	5
2	Выбор занятого временного слота	Слот недоступен для выбора; подтверждение заявки невозможно	Занятый слот заблокирован; выбор и подтверждение недоступны	Успешно
3	Выбор свободного слота и нажатие «Записаться»	Заявка сохранена со статусом «Новая»; отображение в «Мои записи»	Заявка создана успешно; отображается в разделе «Мои записи»	Успешно

Выводы по разделу 5.3.1. Тестирование функции оформления заявки подтвердило корректную работу проверки доступности временных слотов, блокировку занятых интервалов и успешное создание заявки в базе данных со статусом «Новая». После создания заявки риелтору автоматически направляется уведомление о новом запросе на показ.

5.3.2 Тестирование функции публикации отзыва

Клиент может оставить отзыв о риелторе после завершённой записи. Тестовые сценарии и их результаты представлены в таблице 5.6.

Таблица 5.6 – Тестирование функции публикации отзыва о риелторе

Шаг	Действие пользователя	Ожидаемый результат	Фактический результат	Статус
1	2	3	4	5
1	Нажатие «Оставить отзыв» для завершённой записи, ввод оценки и комментария	Отзыв сохранён со статусом «На модерации» (IsApproved = false); форма закрывается	Отзыв создан в БД со статусом IsApproved=false; кнопка отзыва для данной записи стала недоступна	Успешно
2	Повторная попытка оставить отзыв по той же записи	Система запрещает повторную публикацию; кнопка недоступна	Кнопка «Оставить отзыв» для данной записи отсутствует	Успешно
3	Отзыв одобрен администратором (проверка видимости)	Отзыв отображается на публичной странице профиля риелтора	Одобрённый отзыв стал виден на странице риелтора	Успешно

Выводы по разделу 5.3.2. Тестирование функции публикации отзыва подтвердило корректную работу механизма проверки уникальности отзыва на одну запись, передачу отзыва на модерацию администратору и отображение одобренных отзывов на публичной странице риелтора. Отклонённые администратором отзывы не попадают в открытый доступ и остаются видны только автору с пометкой о статусе.

5.3.3 Тестирование функции управления избранным

Клиент может добавлять объекты в личный список избранного и удалять их. Тестовые сценарии и их результаты представлены в таблице 5.7.

Таблица 5.7 – Тестирование функции управления избранным

Шаг	Действие пользователя	Ожидаемый результат	Фактический результат	Статус
1	2	3	4	5
1	Нажатие кнопки «В избранное» на карточке объекта	Объект добавляется в список избранного; счётчик в шапке увеличивается на единицу	Объект добавлен; счётчик обновился	Успешно
2	Повторное нажатие кнопки «В избранное» на том же объекте	Объект удаляется из избранного; счётчик уменьшается на единицу	Объект удалён; счётчик обновился	Успешно
3	Переход в раздел «Избранное»	Отображается список сохранённых объектов с карточками	Список отображается корректно	Успешно
4	Нажатие кнопки удаления объекта в разделе «Избранное»	Объект удаляется из списка без перезагрузки страницы	Объект удалён; список обновился	Успешно

Выводы по разделу 5.3.3. Тестирование функции управления избранным подтвердило корректную работу добавления и удаления объектов, и счетчика.

5.4 Тестирование функций администратора

5.4.1 Тестирование управления ролями пользователей

Администратор может повышать роль клиента до риелтора и выполнять обратное понижение. Тестовые сценарии и их результаты представлены в таблице 5.8.

Таблица 5.8 – Тестирование функции управления ролями пользователей

Шаг	Действие пользователя	Ожидаемый результат	Фактический результат	Статус
1	2	3	4	5
1	Нажатие «Повысить до риелтора» для клиента	Роль пользователя меняется на «Риелтор»; отправляется email-уведомление	Роль обновлена в БД; уведомление отправлено; при следующем входе доступна панель риелтора	Успешно
2	Нажатие «Понизить до клиента» для риелтора	Роль пользователя меняется на «Клиент»	Роль обновлена корректно	Успешно

Продолжение таблицы 5.8 – Тестирование функции управления ролями пользователей

1	2	3	4	5
3	Нажатие «Заблокировать» для активного пользователя	Учётная запись блокируется; пользователь не может войти в систему	Флаг IsBlocked = true; при попытке входа отображается сообщение о блокировке	Успешно

Выводы по разделу 5.4.1. Тестирование подтвердило корректную работу механизмов смены ролей и блокировки учётных записей. После изменения роли пользователь получает доступ к соответствующему интерфейсу при следующей авторизации, а заблокированный пользователь лишается возможности входа в систему.

5.4.2 Тестирование функции модерации отзывов

Администратор может одобрять или отклонять отзывы клиентов перед их публикацией. Тестовые сценарии и их результаты представлены в таблице 5.9.

Таблица 5.9 – Тестирование функции модерации отзывов

Шаг	Действие пользователя	Ожидаемый результат	Фактический результат	Статус
1	2	3	4	5
1	Нажатие «Одобрить» для отзыва, ожидающего модерации	Флаг IsApproved = true; отзыв появляется на странице риелтора	Отзыв одобрен; отображается на публичной странице риелтора	Успешно
2	Нажатие «Отклонить» для отзыва, ожидающего модерации	Отзыв удаляется из БД без публикации	Отзыв удалён физически; на странице риелтора не отображается	Успешно

Выводы по разделу 5.4.2. Тестирование модерации отзывов подтвердило корректную работу механизма предварительной проверки: одобренные отзывы публикуются на странице риелтора, отклонённые удаляются из системы без отображения пользователям.

Проведённое тестирование подтвердило корректную работу всех ключевых функций системы: регистрации и авторизации с разграничением доступа по ролям, управления объектами недвижимости, оформления заявок на показ с проверкой доступности временных слотов, управления избранным, публикации и модерации отзывов, а также административного управления пользователями. Все граничные случаи корректно обрабатываются системой с выводом информативных сообщений пользователю.

6 Руководство по установке и использованию

6.1 Требования к аппаратному и программному обеспечению

Для корректного функционирования программного средства «Агентство недвижимости» рабочее место пользователя должно соответствовать минимальным системным требованиям. Требования к аппаратному обеспечению представлены в таблице 6.1.

Таблица 6.1 – Требования к аппаратному обеспечению

Компонент	Минимальные требования	Рекомендуемые требования
Процессор	x64, 1,5 ГГц	x64, 2,0 ГГц и выше
Оперативная память	512 МБ	2 ГБ и более
Свободное место на диске	200 МБ	500 МБ
Сетевой интерфейс	10 Мбит/с	100 Мбит/с

Требования к программному обеспечению:

- операционная система: Windows 10 (сборка 17763 и выше) или Windows 11;
- среда выполнения .NET Runtime не требует предварительной установки – необходимые компоненты интегрированы в установочный пакет;
- система управления базами данных: Microsoft SQL Server (LocalDB), устанавливается автоматически вместе с приложением.

6.2 Установка программного средства

Развёртывание приложения выполняется с помощью автоматизированного инсталлятора MSM_Setup.exe. Процесс установки состоит из следующих шагов:

- запуск файла MSM_Setup.exe от имени администратора;
- выбор директории для установки (по умолчанию C:\Program Files\MSM\);
- выбор дополнительных параметров: создание ярлыка на рабочем столе;
- подтверждение параметров и запуск установки;
- завершение установки и автоматический запуск приложения.

Для удаления программного средства используется стандартный раздел операционной системы «Параметры» – «Приложения» – «Агентство недвижимости».

6.3 Авторизация и регистрация

Пользователь, впервые заходя в приложение, попадает на главную страницу агентства, доступную без авторизации. Для получения полного доступа к функциям системы необходимо пройти процесс регистрации. На этапе регистрации пользователю нужно ввести следующие данные: логин, полное имя, адрес электронной почты, номер телефона (необязательно),

пароль и подтверждение пароля. Окно регистрации представлено на рисунке 6.1.

Агентство недвижимости
Создание аккаунта

Полное имя *

Логин

E-mail *

Телефон

Пароль

Подтвердите пароль *

Зарегистрироваться

Войти

Рисунок 6.1 – Окно регистрации

При регистрации должны быть выполнены следующие проверки:

- логин и адрес электронной почты должны быть уникальными в системе;
- адрес электронной почты должен быть корректного формата;
- пароль должен содержать не менее 6 символов;
- поле подтверждения пароля должно совпадать с полем пароля.

После успешного завершения регистрации пользователь автоматически перенаправляется на окно авторизации, а на указанный адрес электронной

почты отправляется приветственное уведомление. Интерфейс окна авторизации представлен на рисунке 6.2.

Рисунок 6.2 – Окно авторизации

В окне авторизации пользователь вводит логин и пароль, указанные при регистрации. При успешной проверке учётных данных система перенаправляет пользователя на страницу, соответствующую его роли: клиент попадает в каталог недвижимости, риелтор – в личную панель управления, администратор – в административную панель.

6.4 Руководство по использованию для клиента

После успешной авторизации пользователь с ролью «Клиент» попадает на главный экран с каталогом объектов недвижимости. В верхней панели навигации доступны следующие разделы:

- главная страница – просмотр приветственного экрана агентства;
- каталог – список всех доступных объектов недвижимости с фильтрами;
- избранное – личный список сохранённых объектов;
- мои записи – история заявок на показ;
- личный кабинет – управление профилем;
- о компании – информация об агентстве.

Главная страница показана на рисунке 6.3.

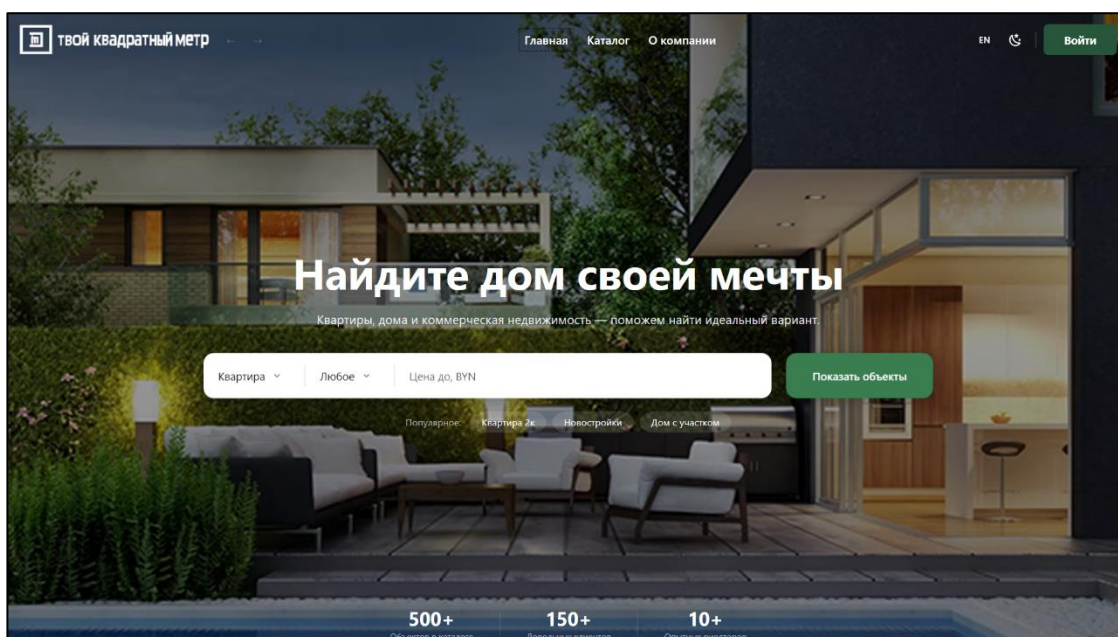


Рисунок 6.3 – Главная страница

При переходе в раздел «Каталог» пользователю отображается список объектов недвижимости в виде карточек. Каждая карточка содержит фотографию объекта, его название, цену, площадь, количество комнат и адрес. На странице каталога доступна гибкая система фильтрации: по цене, площади, количеству комнат, типу объекта, городу и району. Также реализован текстовый поиск по названию и адресу. Страница каталога представлена на рисунке 6.4.

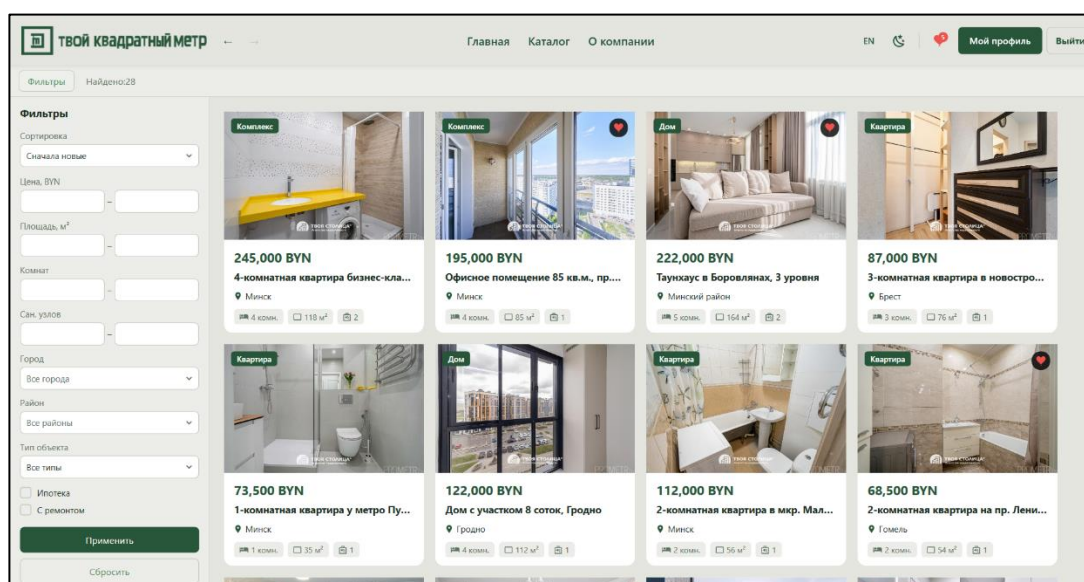


Рисунок 6.4 – Каталог объектов недвижимости

При нажатии на карточку объекта открывается страница с его детальным описанием. На данной странице отображаются: фотогалерея с каруселью изображений, полные технические характеристики (площадь, количество комнат, этаж, год постройки, тип объекта и др.), описание, цена, а также

история изменения цены. Здесь же расположена кнопка добавления объекта в «Избранное» и форма оформления заявки на показ. Страница детального просмотра объекта представлена на рисунке 6.5.

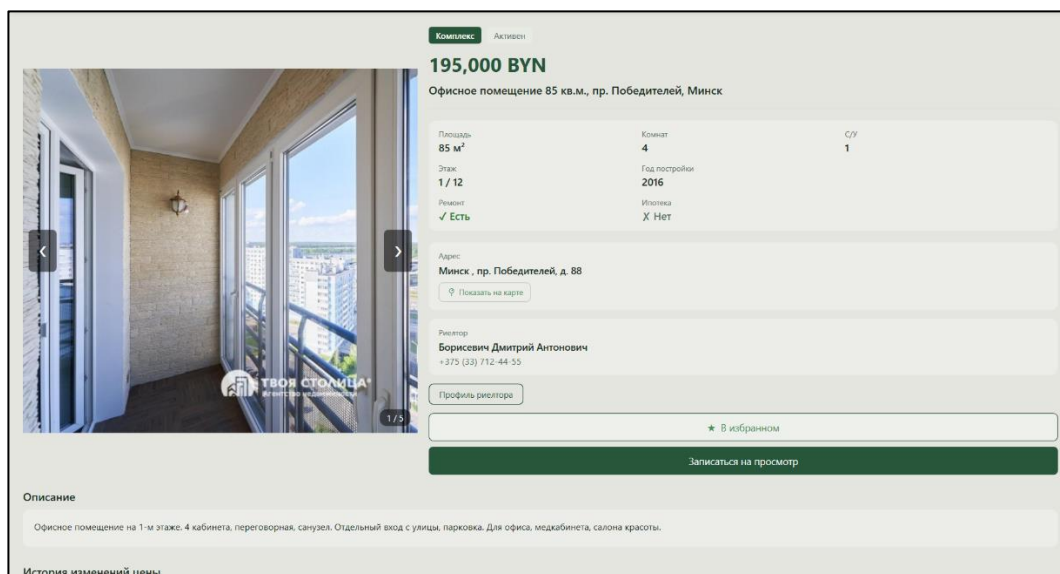


Рисунок 6.5 – Страница детального просмотра объекта

Для оформления заявки на показ клиент выбирает удобный временной слот из доступного расписания риелтора, при необходимости добавляет комментарий и нажимает кнопку подтверждения. После отправки заявки она сохраняется в системе со статусом «Новая» и отображается в разделе «Мои записи». Форма оформления заявки представлена на рисунке 6.6.

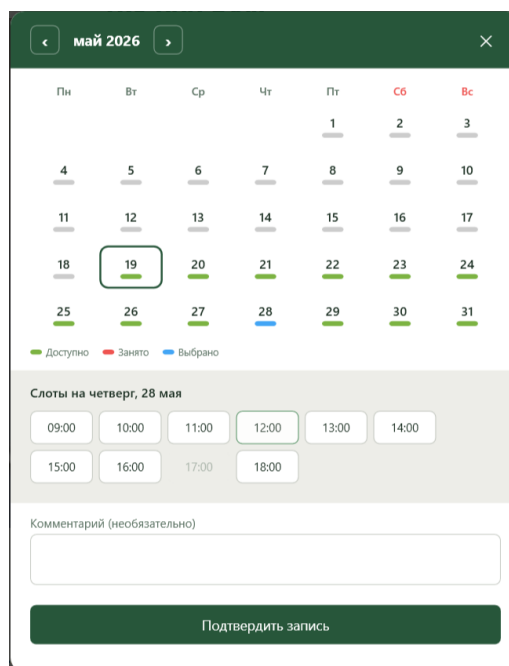


Рисунок 6.6 – Форма оформления заявки на показ

В разделе «Мои записи» клиент может просматривать все свои заявки с текущим статусом («Новая», «Подтверждена», «Завершена», «Отменена»), а

также отменять заявки, находящиеся в статусе «Новая» или «Подтверждена». После завершённой записи клиенту становится доступна возможность оставить отзыв о риелторе. Страница «Мои записи» представлена на рисунке 6.7.

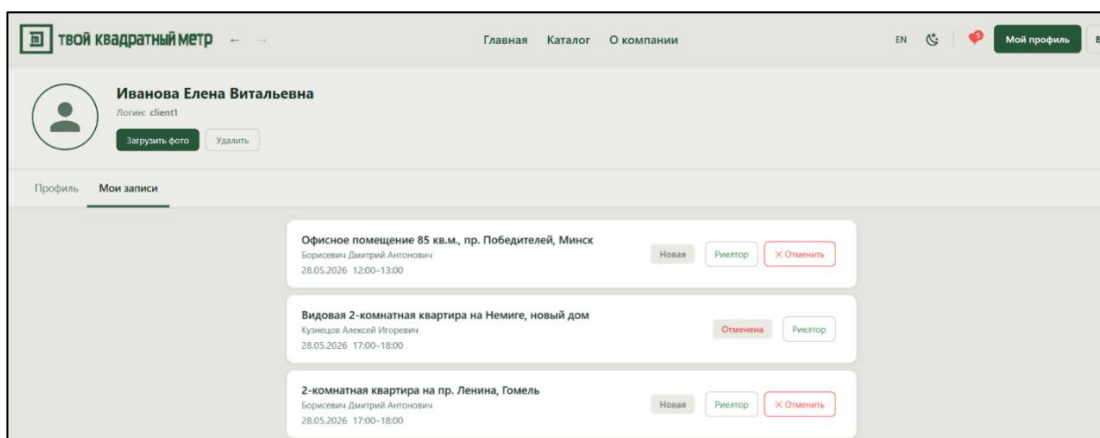


Рисунок 6.7 – Страница «Мои записи»

В разделе «Личный кабинет» клиент может просматривать и редактировать свои персональные данные: полное имя, номер телефона, описание и фотографию профиля, а также изменить пароль. Страница личного кабинета клиента представлена на рисунке 6.8.

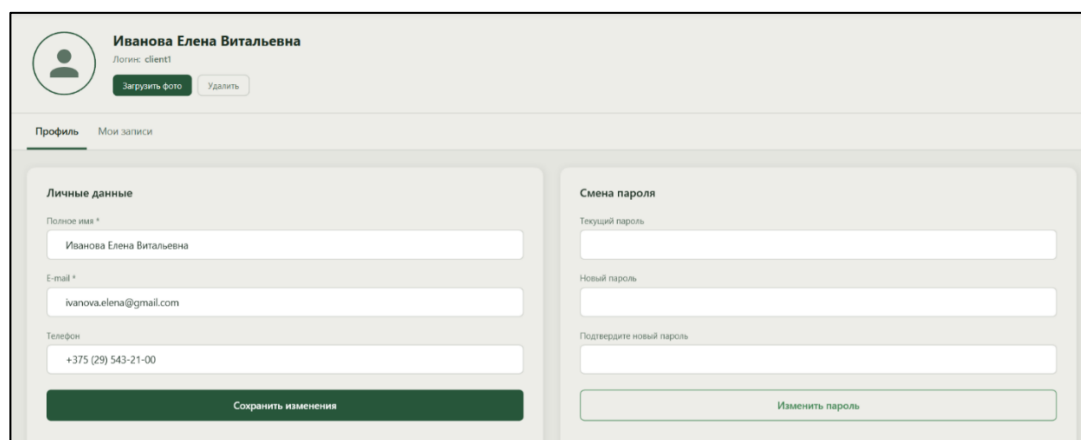


Рисунок 6.8 – Личный кабинет клиента

6.5 Руководство по использованию для риелтора

После авторизации пользователь с ролью «Риелтор» попадает в личную панель управления. Панель содержит следующие вкладки:

- мои объекты – управление объектами недвижимости: просмотр, добавление, редактирование и удаление;
- заявки – список входящих заявок клиентов с инструментами смены статуса;
- расписание – управление личным календарём показов и блокировкой временных интервалов;

- статистика – персональные показатели: количество объектов, заявок, средний рейтинг;
 - профиль – редактирование личных данных риелтора.
- Главная страница панели риелтора представлена на рисунке 6.9.

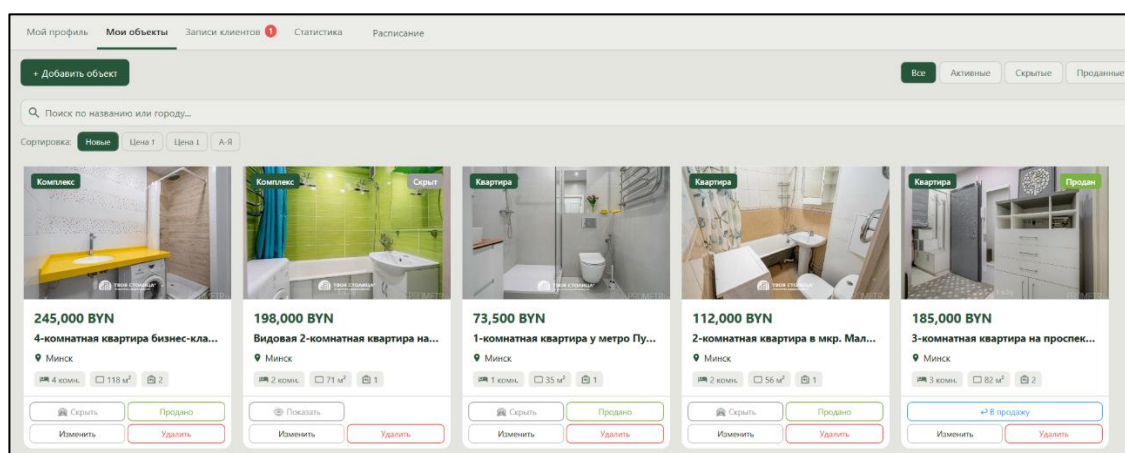


Рисунок 6.9 – Панель риелтора, вкладка «Мои объекты»

На вкладке «Мои объекты» риелтор видит список своих объектов. При нажатии на кнопку добавления открывается форма с полями для ввода характеристик нового объекта и загрузки фотографий. При нажатии на объект из списка риелтор может редактировать его описание, цену, характеристики, а также управлять фотографиями и статусом объекта.

На вкладке «Заявки» отображается список всех входящих запросов от клиентов. Риелтор может подтвердить заявку, отменить её или отметить, как завершённую. При изменении статуса клиент автоматически получает уведомление по электронной почте. Вкладка управления заявками представлена на рисунке 6.10.

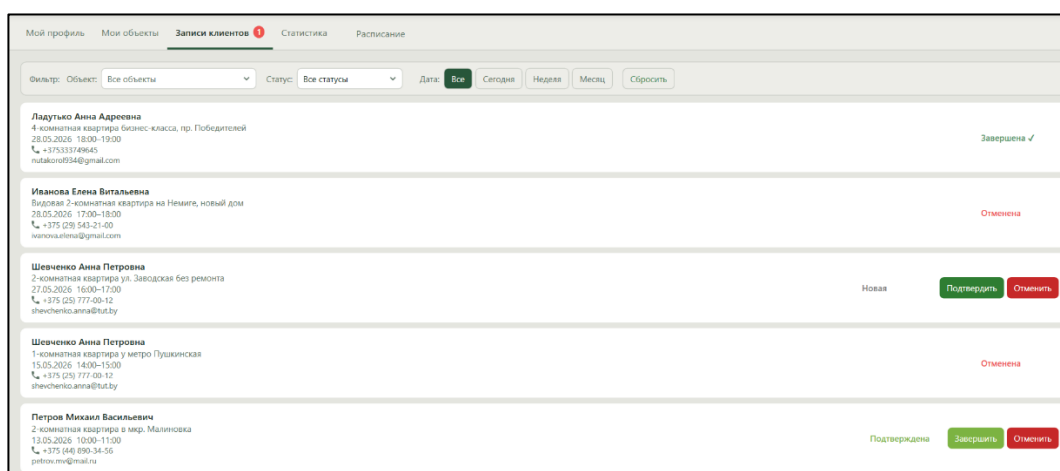


Рисунок 6.10 – Панель риелтора, вкладка «Заявки»

На вкладке «Статистика» риелтор видит сводку своей активности: количество активных объектов, общее количество заявок в разбивке по

статусам, средний рейтинг по отзывам клиентов, а также графики динамики заявок по месяцам. Вкладка статистики представлена на рисунке 6.11.

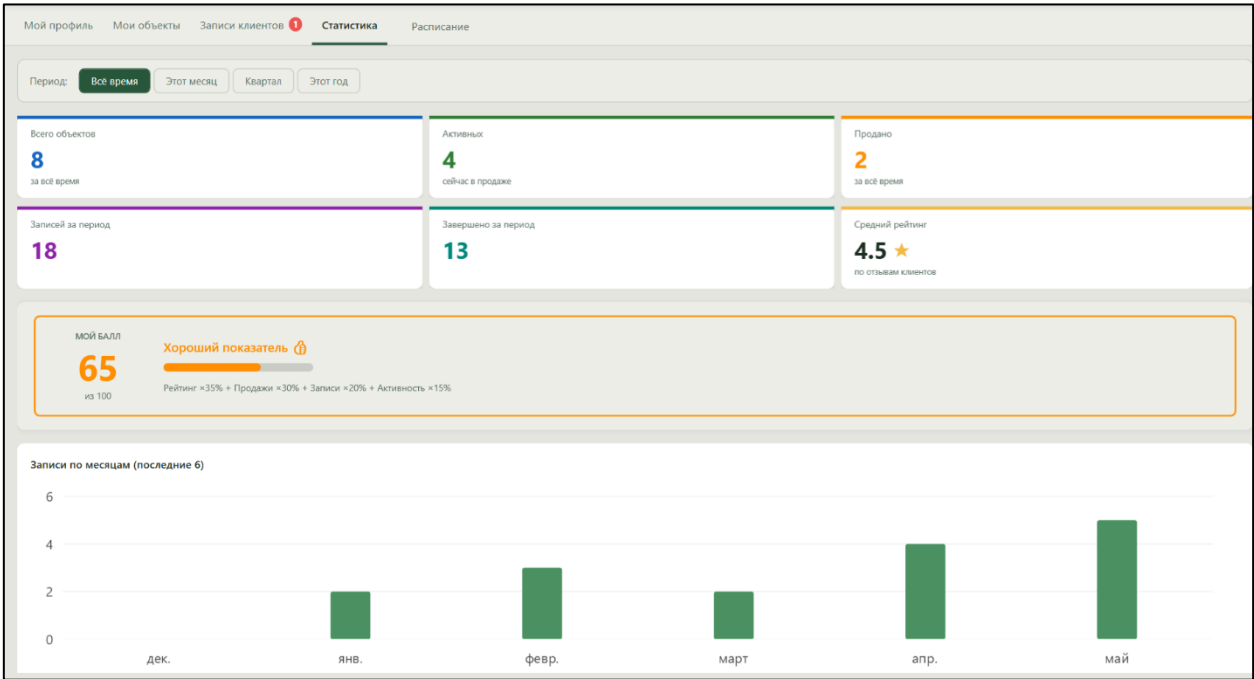


Рисунок 6.11 – Панель риелтора, вкладка «Статистика»

На вкладке «Расписание» риелтор видит календарь с отображением количества записей на каждый день. Вкладка расписания представлена на рисунке 6.12.

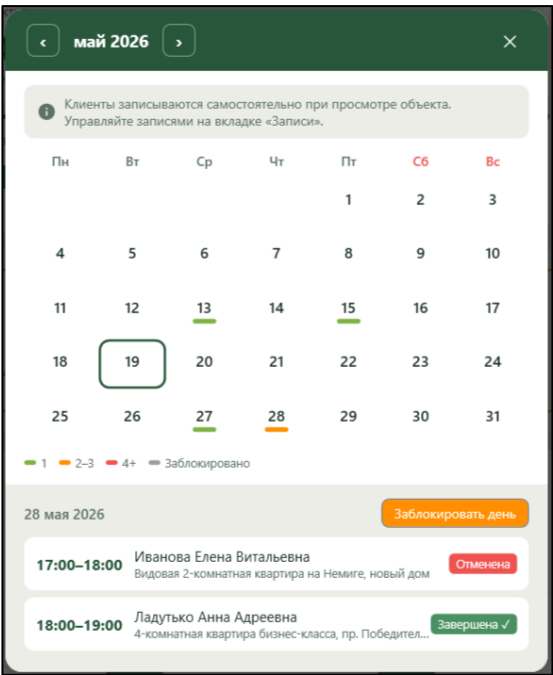


Рисунок 6.12 – Панель риелтора, вкладка «Расписание»

Данная вкладка позволяет оперативно оценить загруженность и спланировать рабочее время.

6.6 Руководство по использованию для администратора

После авторизации пользователь с ролью «Администратор» попадает в административную панель управления. Панель содержит следующие вкладки:

- пользователи – управление учётными записями: просмотр всех пользователей, смена ролей, блокировка;
- объекты – глобальный контроль над каталогом недвижимости агентства;
- статистика – сводная аналитика по всему агентству;
- модерация – проверка и одобрение отзывов клиентов;
- рассылка – отправка информационных писем клиентам и риелторам;
- профиль – редактирование данных администратора.

Административная панель, вкладка «Пользователи» представлена на рисунке 6.13.

Мой профиль Пользователи Модерация отзывов Статистика Рассылка Объекты					
Всё Риелторы Клиенты Заблокированные					
Имя	Логин	E-mail	Роль	Создан	Действия
Краснова Ирина Викторовна	admin	admin@belagency.by	Администратор	08.11.2024	Блокировать
Жук Игорь Павлович	client7	zhuk.igor@tut.by	Клиент	01.11.2025	Риелтор Блокировать
Иванова Елена Витальевна	client1	ivanova.elena@gmail.com	Клиент	08.09.2025	Риелтор Блокировать
Климович Андрей Дмитриевич	client5	klimovich.ad@mail.ru	Клиент	18.04.2026	Риелтор Блокировать
Ладутко Анна Адиевна	klywxq	nutakorol934@gmail.com	Клиент	10.05.2026	Риелтор Блокировать
Петров Михаил Васильевич	client2	petrov.mv@mail.ru	Клиент	08.12.2025	Риелтор Блокировать
Романов Сергей Николаевич	client3	romanov.sn@yandex.by	Клиент	08.02.2026	Риелтор Блокировать
Соколова Татьяна Александровна	client6	sokolova.ta@gmail.com	Клиент	15.01.2026	Риелтор Блокировать
Толик Блатной	tolik123	donig2007@gmail.com	Клиент	18.05.2026	Риелтор Блокировать
Шевченко Анна Петровна	client4	shevchenko.anna@tut.by	Клиент	08.03.2026	Риелтор Блокировать
Борисевич Дмитрий Антонович	realtor3	borisevich@belagency.by	Риелтор	08.10.2025	Клиент Блокировать
Захарова Людмила Николаевна	realtor7	zakharova@belagency.by	Риелтор	15.10.2025	Клиент Блокировать
Козловская Марина Ивановна	realtor5	kozlovskaya@belagency.by	Риелтор	15.09.2025	Клиент Блокировать
Кузнецов Алексей Игоревич	realtor1	kuznetsov@belagency.by	Риелтор	08.03.2025	Клиент Блокировать
Лихачёва Наталья Сергеевна	realtor2	likhacheva@belagency.by	Риелтор	08.07.2025	Клиент Блокировать
Петровский Андрей Семёнович	realtor6	petrovskiy@belagency.by	Риелтор	01.10.2025	Клиент Блокировать
Сидоров Павел Романович	realtor4	sidorov@belagency.by	Риелтор	10.09.2025	Клиент Блокировать
Кирилл	kirill8642	kirill@tello@gmail.com	Клиент	08.05.2026	Блокировать

Рисунок 6.13 – Административная панель, вкладка «Пользователи»

На вкладке «Пользователи» администратор видит список всех зарегистрированных аккаунтов с возможностью фильтрации по роли. Для пользователя доступны кнопки повышения до риелтора, понижения до клиента, а также блокировки или разблокировки учётной записи. При блокировке система запрашивает подтверждение и предлагает указать причину. Заблокированный пользователь теряет доступ к системе, но его данные сохраняются в базе.

На вкладке «Модерация» администратор проверяет отзывы, ожидающие публикации. Для каждого отзыва отображается имя клиента, текст, оценка и имя риелтора, которому оставлен отзыв. Администратор может одобрить отзыв для публикации или отклонить его с указанием причины. Одобренные отзывы сразу появляются на странице риелтора и влияют на его рейтинг. Вкладка модерации представлена на рисунке 6.14.

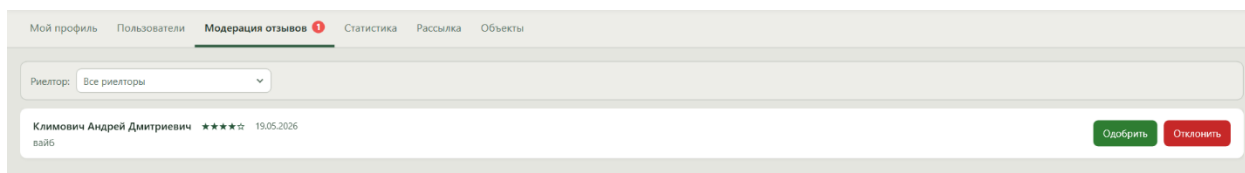


Рисунок 6.14 – Административная панель, вкладка «Модерация»

На вкладке «Рассылка» администратор может составить письмо и отправить его выбранной группе получателей: всем клиентам, всем риелторам или всем пользователям системы. Вкладка рассылки представлена на рисунке 6.15.

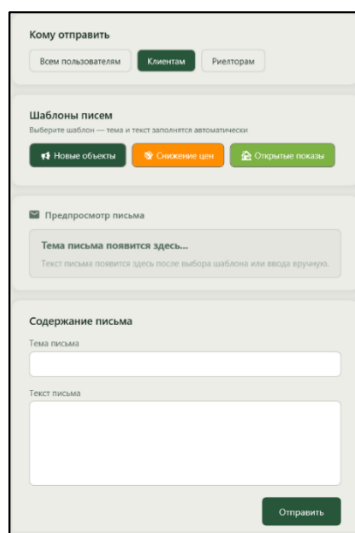


Рисунок 6.15 – Административная панель, вкладка «Рассылка»

На вкладке «Статистика» администратор видит сводную аналитику по всему агентству: общее количество объектов, пользователей, заявок, новых клиентов за период, а также таблицу с показателями каждого риелтора (количество объектов, заявок и средний рейтинг). Вкладка статистики представлена на рисунке 6.16.

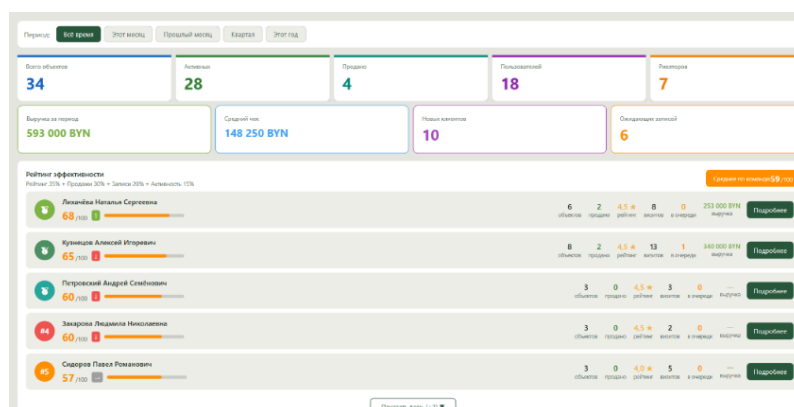


Рисунок 6.16 – Административная панель, вкладка «Статистика»

Административная панель предоставляет полный набор инструментов для управления агентством: контроль над пользователями и их ролями,

модерация контента, массовые рассылки и мониторинг эффективности работы. Интерфейс обеспечивает удобный и быстрый доступ ко всем функциям управления системой.

В данном разделе представлено руководство по использованию программного средства для каждой роли пользователя. Описан процесс регистрации и авторизации, работа клиента с каталогом, избранным и заявками на показ, функции риелтора по управлению объектами и обработке заявок, а также инструменты администратора для управления пользователями, модерации отзывов и массовых рассылок. Интерфейс каждого раздела проиллюстрирован скриншотами.

Заключение

В рамках выполнения курсового проекта было разработано и реализовано программное средство «Агентство недвижимости», предназначенное для автоматизации работы агентства и обеспечения взаимодействия пользователей с различными ролями: клиент, риелтор и администратор.

В ходе работы:

- изучены подходы к построению настольных клиентских приложений с использованием технологии WPF и архитектурного паттерна MVVM;
- исследована организация реляционных баз данных и реализована структура хранения данных в Microsoft SQL Server с применением Entity Framework Core по принципу Code First;
- разработана модель ролевого доступа с учётом функциональных различий между клиентом, риелтором и администратором;
- реализованы пользовательские интерфейсы для всех ролей: каталог недвижимости с системой фильтрации, страница детального просмотра объекта, личный кабинет клиента с управлением заявками и избранным;
- создана панель риелтора с функциями управления объектами недвижимости, обработки заявок, ведения расписания показов и просмотра личной статистики;
- реализована административная панель с инструментами управления ролями пользователей, модерации отзывов, массовой email-рассылки и просмотра сводной аналитики агентства;
- интегрирована система автоматических email-уведомлений через SMTP-сервер для оповещения пользователей о ключевых событиях в системе.

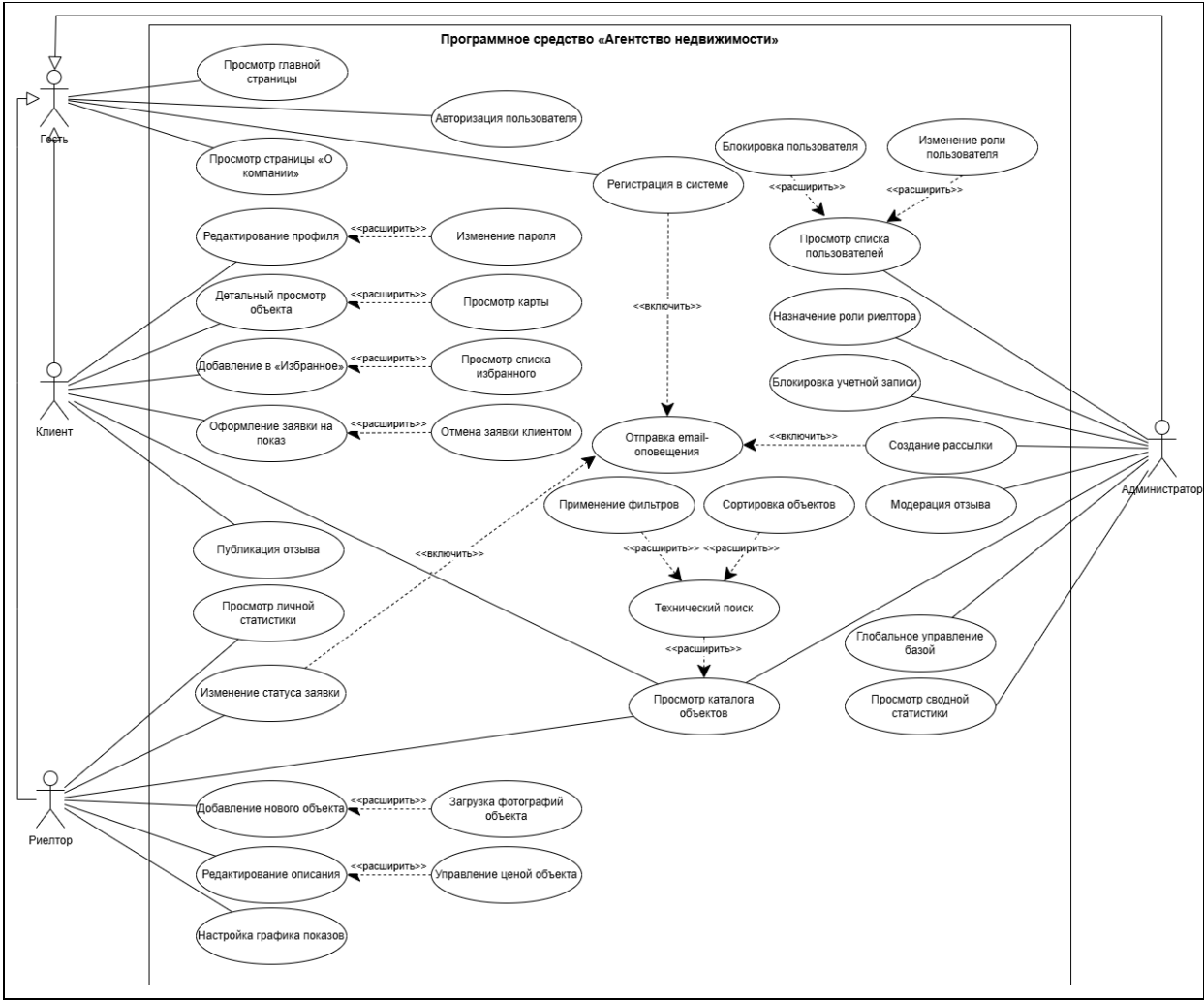
Проект демонстрирует полноценную модель информационной системы агентства недвижимости и может служить основой для дальнейшего развития. В перспективе возможно расширение приложения за счёт внедрения клиент-серверной архитектуры, разработки веб-версии или мобильного приложения, а также интеграции с картографическими сервисами для отображения объектов на карте.

Список использованных источников

- 1 Яндекс Недвижимость [Электронный ресурс] – Режим доступа: <https://realty.ya.ru/> – Дата доступа: 14.03.2026.
- 2 Домклик [Электронный ресурс] – Режим доступа: <https://domclick.ru/> – Дата доступа: 14.03.2026.
- 3 Циан [Электронный ресурс] – Режим доступа: <https://www.cian.ru/> – Дата доступа: 14.03.2026.
- 4 Руководство по WPF [Электронный ресурс] – Режим доступа: <https://metanit.com/sharp/wpf/> – Дата доступа: 20.04.2026.
- 5 Документация Entity Framework Core [Электронный ресурс] – Режим доступа: <https://learn.microsoft.com/ru-ru/ef/> – Дата доступа: 22.04.2026.
- 6 MVVM полное понимание (+WPF) [Электронный ресурс] – Режим доступа: <https://habr.com/ru/articles/338518/> – Дата доступа: 25.04.2026.
- 7 Курс лекций по ООП Белодед Н.И.

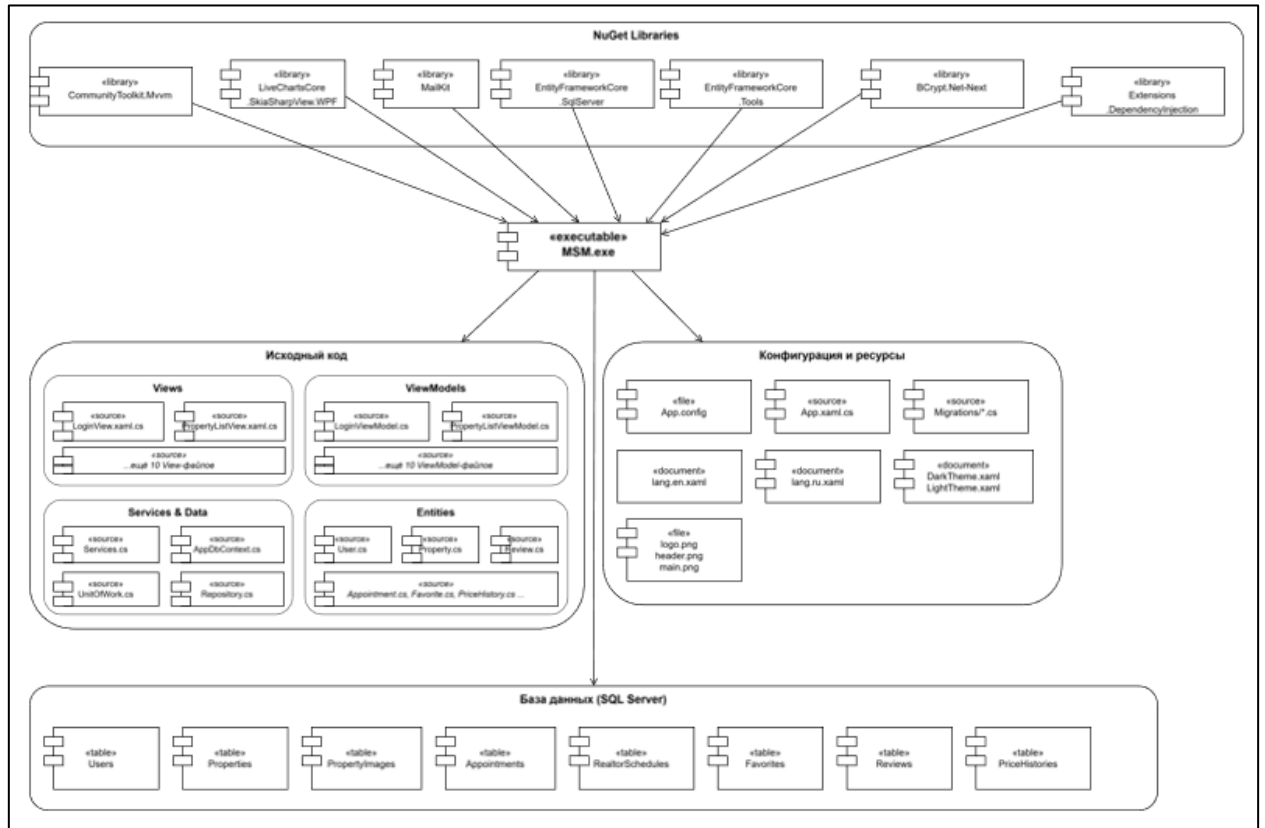
Приложение А

Диаграмма вариантов использования программного средства



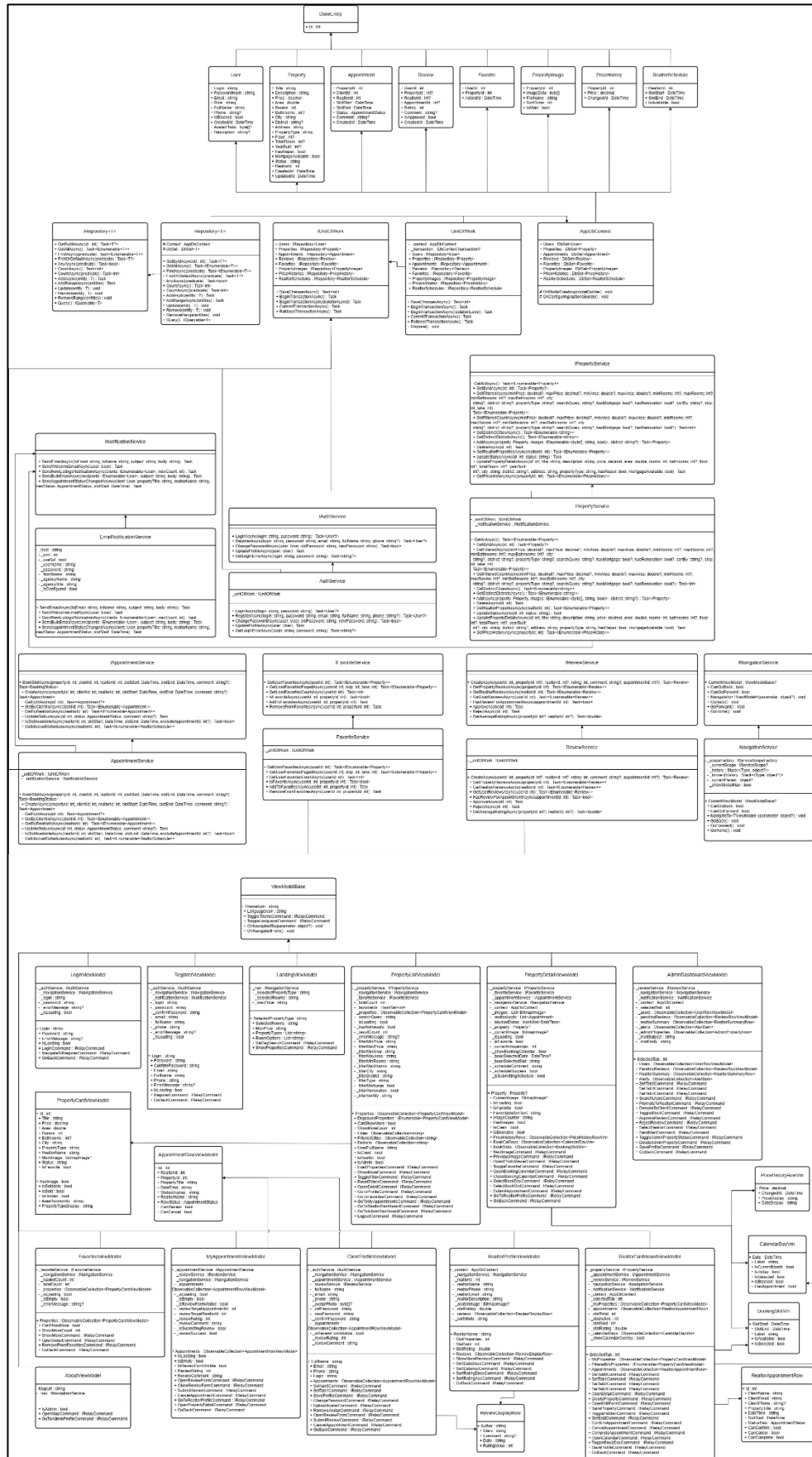
Приложение Б

Диаграмма компонентов программного средства



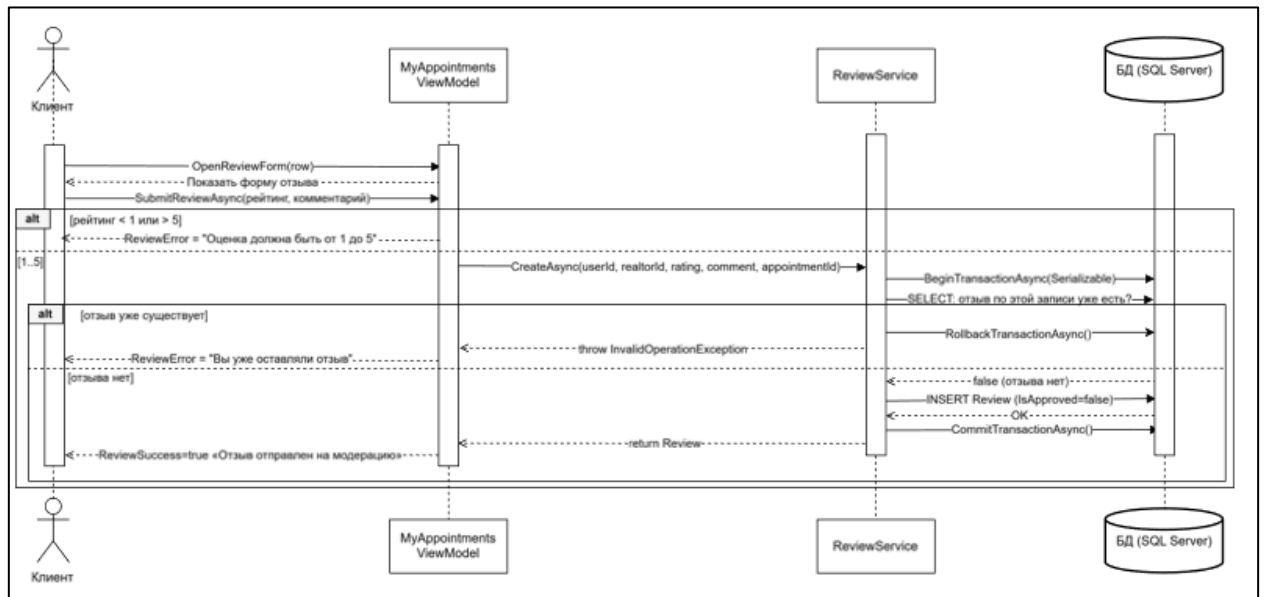
Приложение В

Диаграмма классов программного средства



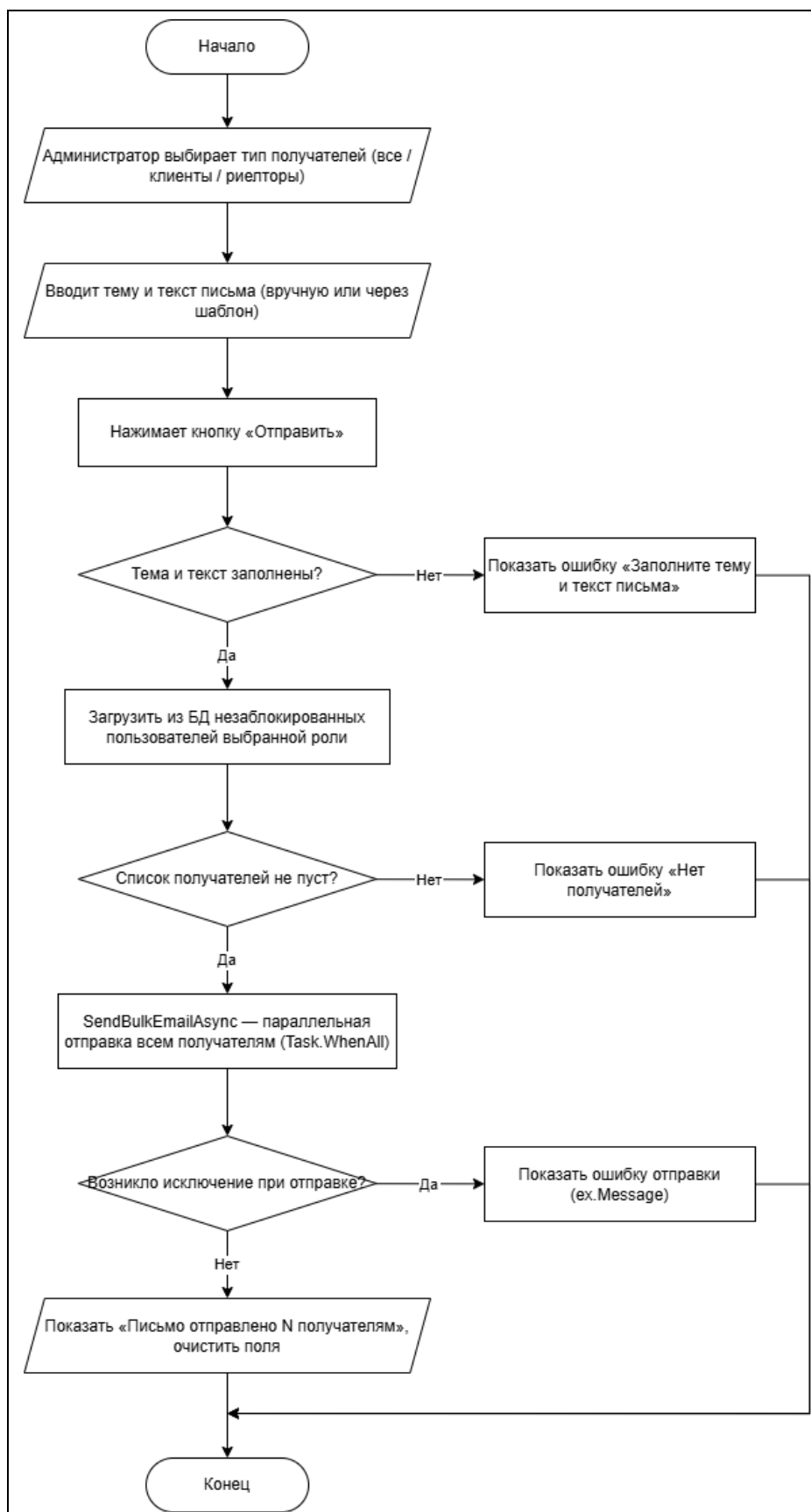
Приложение Г

Диаграмма последовательности взаимодействия



Приложение Д

Блок-схема алгоритма регистрации пользователя



Приложение Е

Листинг – Функция регистрации пользователя

```
public async Task<User?> RegisterAsync(string login, string
password,
    string email, string fullName, string? phone = null)
{
    if (await _unitOfWork.Users.AnyAsync(u => u.Login ==
login))
        return null;

    if (await _unitOfWork.Users.AnyAsync(u => u.Email ==
email))
        return null;

    var user = new User
    {
        Login          = login,
        PasswordHash   = BCrypt.Net.BCrypt.HashPassword(pass-
word),
        Email          = email,
        FullName       = fullName,
        Phone          = phone,
        Role           = "client",
        CreatedAt      = DateTime.Now
    };

    await _unitOfWork.Users.AddAsync(user);
    await _unitOfWork.SaveChangesAsync();
    return user;
}
```

Приложение Ж

Листинг – Функция авторизации пользователя

```
public async Task<User?> LoginAsync(string login, string password)
{
    var user = await _unitOfWork.Users
        .FirstOrDefaultAsync(u => u.Login == login);
    if (user == null) return null;

    if (!BCrypt.Net.BCrypt.Verify(password, user.PasswordHash))
        return null;

    if (user.IsBlocked) return null;

    return user;
}

public async Task<string?> GetLoginErrorAsync(string login,
string password)
{
    var user = await _unitOfWork.Users
        .FirstOrDefaultAsync(u => u.Login == login);
    if (user == null)
        return "Неверный логин или пароль.";
    if (!BCrypt.Net.BCrypt.Verify(password, user.PasswordHash))
        return "Неверный логин или пароль.";
    if (user.IsBlocked)
        return "Аккаунт заблокирован. Обратитесь к администратору.";
    return null;
}
```

Приложение 3

Листинг – Функция добавления объекта недвижимости

```

public async Task<Property> AddAsync(Property property,
    IEnumerable<(byte[] Data, string FileName, bool IsMain)>
images,
    string? district = null)
{
    property.District = string.IsNullOrEmpty(district)
        ? null : district.Trim();
    property.CreatedAt = DateTime.Now;
    property.UpdatedAt = DateTime.Now;
    property.Status = "active";

    await _unitOfWork.BeginTransactionAsync();
    try
    {
        await _unitOfWork.Properties.AddAsync(property);
        await _unitOfWork.SaveChangesAsync();

        await _unitOfWork.PriceHistories.AddAsync(new Price-
History
        {
            PropertyId = property.Id,
            Price = property.Price,
            ChangedAt = property.CreatedAt
        });
        await _unitOfWork.SaveChangesAsync();

        var imageList = images.Select((img, index) => new
PropertyImage
        {
            PropertyId = property.Id,
            ImageData = img.Data,
            FileName = img.FileName,
            IsMain = img.IsMain,
            SortOrder = index
        }).ToList();

        await _unitOfWork.PropertyImages.AddRangeAsync(image-
List);

        await _unitOfWork.SaveChangesAsync();
        await _unitOfWork.CommitTransactionAsync();
    }
    catch
    {
        await _unitOfWork.RollbackTransactionAsync();
        throw;
    }

    var activeCount = await _unitOfWork.Properties.Query()
        .CountAsync(p => p.Status == "active");

```

```
if (activeCount > 0 && activeCount % 10 == 0)
{
    var clients = await _unitOfWork.Users.Query()
        .Where(u => u.Role == "client"
            && !u.IsBlocked
            && u.Email != null)
        .ToListAsync();
    _ = _notificationService
        .SendNewListingsNotificationAsync(clients, ac-
tiveCount);
}
return property;
}
```

Приложение И

Листинг – Функция оформления заявки на показ

```

public async Task<BookingStatus> BookSlotAsync(int propertyId,
    int clientId, int realtorId,
    DateTime slotStart, DateTime slotEnd, string? comment =
null)
{
    await _unitOfWork.BeginTransactionAsync(
        System.Data.IsolationLevel.Serializable);
    try
    {
        var duplicate = await _unitOfWork.Appointments.An-
yAsync(a =>
            a.ClientId == clientId &&
            a.PropertyId == propertyId &&
            (a.Status == AppointmentStatus.New ||
            a.Status == AppointmentStatus.Confirmed));
        if (duplicate)
        {
            await _unitOfWork.RollbackTransactionAsync();
            return BookingStatus.AlreadyBooked;
        }

        var conflict = await _unitOfWork.Appointments.An-
yAsync(a =>
            a.RealtorId == realtorId &&
            a.Status != AppointmentStatus.Cancelled &&
            a.SlotStart < slotEnd &&
            a.SlotEnd > slotStart);
        if (conflict)
        {
            await _unitOfWork.RollbackTransactionAsync();
            return BookingStatus.SlotTaken;
        }

        var appointment = new Appointment
        {
            PropertyId = propertyId,
            ClientId = clientId,
            RealtorId = realtorId,
            SlotStart = slotStart,
            SlotEnd = slotEnd,
            Comment = comment,
            Status = AppointmentStatus.New,
            CreatedAt = DateTime.Now
        };
        await _unitOfWork.Appointments.AddAsync(appointment);
        await _unitOfWork.SaveChangesAsync();
        await _unitOfWork.CommitTransactionAsync();
        return BookingStatus.Success;
    }
}

```

```
        catch
        {
            await _unitOfWork.RollbackTransactionAsync();
            throw;
        }
    }
```

Приложение К

Листинг – Реализация класса AppDbContext

```

using Microsoft.EntityFrameworkCore;
using MSM.Models.Entities;

namespace MSM.Data.Context;

public class AppDbContext : DbContext
{
    public AppDbContext()
    {
    }

    public AppDbContext(DbContextOptions<AppDbContext> options)
    : base(options)
    {
    }

    public DbSet<User> Users => Set<User>();
    public DbSet<Property> Properties => Set<Property>();
    public DbSet<PropertyImage> PropertyImages =>
Set<PropertyImage>();
    public DbSet<Appointment> Appointments =>
Set<Appointment>();
    public DbSet<RealtorSchedule> RealtorSchedules =>
Set<RealtorSchedule>();
    public DbSet<Favorite> Favorites => Set<Favorite>();
    public DbSet<Review> Reviews => Set<Review>();
    public DbSet<PriceHistory> PriceHistories =>
Set<PriceHistory>();

    protected override void OnModelCreating(ModelBuilder
modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        // User configuration
        modelBuilder.Entity<User>(entity =>
        {
            entity.HasKey(e => e.Id);
            entity.HasIndex(e => e.Login).IsUnique();
            entity.HasIndex(e => e.Email).IsUnique();
            entity.Property(e =>
e.Login).HasMaxLength(50).IsRequired();
            entity.Property(e =>
e.PasswordHash).HasMaxLength(255).IsRequired();
            entity.Property(e =>
e.Email).HasMaxLength(100).IsRequired();
            entity.Property(e =>
e.Role).HasMaxLength(20).IsRequired();
            entity.Property(e =>

```

```

e.FullName).HasMaxLength(100).IsRequired();
    entity.Property(e => e.Phone).HasMaxLength(20);
});

// Property configuration
modelBuilder.Entity<Property>(entity =>
{
    entity.HasKey(e => e.Id);
    entity.Property(e =>
e.Title).HasMaxLength(200).IsRequired();
    entity.Property(e => e.Description).IsRequired();
    entity.Property(e => e.Price).IsRequired();
    entity.Property(e => e.Area).IsRequired();
    entity.Property(e => e.Rooms).IsRequired();
    entity.Property(e =>
e.City).HasMaxLength(100).IsRequired();
    entity.Property(e => e.District).HasMaxLength(100);
    entity.Property(e =>
e.Address).HasMaxLength(300).IsRequired();
    entity.Property(e =>
e.PropertyType).HasMaxLength(20).IsRequired();
    entity.Property(e =>
e.Status).HasMaxLength(20).IsRequired().HasDefaultValue("active"
);
    entity.HasOne(e => e.Realtor)
        .WithMany(e => e.Properties)
        .HasForeignKey(e => e.RealtorId)
        .OnDelete(DeleteBehavior.Restrict);
    entity.HasIndex(e => e.Price);
    entity.HasIndex(e => e.City);
});

// PropertyImage configuration
modelBuilder.Entity<PropertyImage>(entity =>
{
    entity.HasKey(e => e.Id);
    entity.Property(e => e.ImageData).IsRequired();
    entity.Property(e =>
e.FileName).HasMaxLength(255).IsRequired();
    entity.HasOne(e => e.Property)
        .WithMany(e => e.Images)
        .HasForeignKey(e => e.PropertyId)
        .OnDelete(DeleteBehavior.Cascade);
});

// Appointment configuration
modelBuilder.Entity<Appointment>(entity =>
{
    entity.HasKey(e => e.Id);
    entity.Property(e => e.Status)

.HasMaxLength(20).IsRequired().HasDefaultValueSql("'new'")
        .HasConversion(

```

```

        v => v.ToString().ToLower(),
        v => Enum.Parse<AppointmentStatus>(v,
true));
        entity.Property(e => e.Comment).HasMaxLength(500);
        // Запрещаем двойное бронирование одного слота у
риелтора (кроме отменённых)
        entity.HasIndex(e => new { e.RealtorId, e.SlotStart
}))
            .HasFilter("[Status] != 'cancelled'")
            .IsUnique();

.HasDatabaseName("IX_Appointments_RealtorId_SlotStart_Active");
entity.HasOne(e => e.Property)
    .WithMany(e => e.Appointments)
    .HasForeignKey(e => e.PropertyId)
    .OnDelete(DeleteBehavior.Restrict);
entity.HasOne(e => e.Client)
    .WithMany(e => e.ClientAppointments)
    .HasForeignKey(e => e.ClientId)
    .OnDelete(DeleteBehavior.Restrict);
entity.HasOne(e => e.Realtor)
    .WithMany(e => e.RealtorAppointments)
    .HasForeignKey(e => e.RealtorId)
    .OnDelete(DeleteBehavior.Restrict);
});

// RealtorSchedule configuration
modelBuilder.Entity<RealtorSchedule>(entity =>
{
    entity.HasKey(e => e.Id);
    entity.Property(e =>
e.IsAvailable).HasDefaultValue(true);
    entity.HasOne(e => e.Realtor)
        .WithMany(e => e.Schedules)
        .HasForeignKey(e => e.RealtorId)
        .OnDelete(DeleteBehavior.Restrict);
});

// Favorite configuration
modelBuilder.Entity<Favorite>(entity =>
{
    entity.HasKey(e => e.Id);
    entity.HasIndex(e => new { e.UserId, e.PropertyId
}).IsUnique();
    entity.HasOne(e => e.User)
        .WithMany(e => e.Favorites)
        .HasForeignKey(e => e.UserId)
        .OnDelete(DeleteBehavior.Restrict);
    entity.HasOne(e => e.Property)
        .WithMany(e => e.Favorites)
        .HasForeignKey(e => e.PropertyId)
        .OnDelete(DeleteBehavior.Restrict);
});

```

```

// PriceHistory configuration
modelBuilder.Entity<PriceHistory>(entity =>
{
    entity.HasKey(e => e.Id);
    entity.Property(e => e.Price).IsRequired();
    entity.HasOne(e => e.Property)
        .WithMany()
        .HasForeignKey(e => e.PropertyId)
        .OnDelete(DeleteBehavior.Cascade);
    entity.HasIndex(e => e.PropertyId);
});

// Review configuration
modelBuilder.Entity<Review>(entity =>
{
    entity.HasKey(e => e.Id);
    entity.Property(e =>
e.IsApproved).HasDefaultValue(false);
    // Один отзыв на одну запись на просмотр
    entity.HasIndex(e => e.AppointmentId)
        .IsUnique()
        .HasFilter("[AppointmentId] IS NOT NULL")

.HasDatabaseName("IX_Reviews_AppointmentId_Unique");
    entity.HasOne(e => e.User)
        .WithMany(e => e.Reviews)
        .HasForeignKey(e => e.UserId)
        .OnDelete(DeleteBehavior.Restrict);
    entity.HasOne(e => e.Property)
        .WithMany(e => e.Reviews)
        .HasForeignKey(e => e.PropertyId)
        .OnDelete(DeleteBehavior.Restrict);
    entity.HasOne(e => e.Realtor)
        .WithMany()
        .HasForeignKey(e => e.RealtorId)
        .OnDelete(DeleteBehavior.Restrict);
});
}

protected override void
OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    if (!optionsBuilder.IsConfigured)
    {
        optionsBuilder.UseSqlServer(

System.Configuration.ConfigurationManager.ConnectionStrings["RealEstateConnection"]?.ConnectionString
        ??
"Server=(localdb)\\mssqllocaldb;Database=RealEstateDb;Trusted_Connection=True;MultipleActiveResultSets=true;Encrypt=False");
    }
}

```

}	}
---	---